

**Kompensation des Overestimation Bias im
Reinforcement Learning: Ein Vergleich von
DQN und DDQN in einer Rennspielumgebung**

Manuel Wirth

Bachelor-Thesis

zur Erlangung des akademischen Grades Bachelor of Science (B.Sc.)

Studiengang Informatik

Fakultät für Informatik

Technische Hochschule Mannheim

05.08.2025

Betreuer

Prof. Dr. Jörn Fischer, Technische Hochschule Mannheim

Prof. Dr. Thomas Ihme, Technische Hochschule Mannheim

Wirth, Manuel:

Kompensation des Overestimation Bias im Reinforcement Learning: Ein Vergleich von DQN und DDQN in einer Rennspielumgebung / Manuel Wirth. –

Bachelor-Thesis, Mannheim: Technische Hochschule Mannheim, 2025. [58](#) Seiten.

Wirth, Manuel:

Compensation of overestimation bias in reinforcement learning: A comparison of DQN and DDQN in a racing game environment / Manuel Wirth. –

Bachelor Thesis, Mannheim: University of Applied Sciences Mannheim, 2025. [58](#) pages.

Um die Lesbarkeit zu vereinfachen, wird auf die zusätzliche Formulierung der weiblichen Form bei Personenbezeichnungen verzichtet. Ich weise deshalb darauf hin, dass die Verwendung der männlichen Form explizit als geschlechtsunabhängig verstanden werden soll.

Erklärung

Hiermit erkläre ich, dass ich die vorliegende Arbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe.

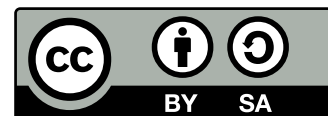
Mannheim, 05.08.2025



Manuel Wirth

Ich bin damit einverstanden, dass meine Arbeit veröffentlicht wird, d. h. dass die Arbeit elektronisch gespeichert, in andere Formate konvertiert, auf den Servern der Technische Hochschule Mannheim öffentlich zugänglich gemacht und über das Internet verbreitet werden darf.

Dieses Werk ist lizenziert unter einer [Creative Commons](#) „Namensnennung – Weitergabe unter gleichen Bedingungen 4.0 International“ Lizenz.



Abstrakt

Kompensation des Overestimation Bias im Reinforcement Learning: Ein Vergleich von DQN und DDQN in einer Rennspielumgebung

Der „Overestimation Bias“ ist eine bekannte Schwäche des Deep Q-Network (DQN) Algorithmus, die zu instabilem Training führen kann. Als Lösung wurde der Double Deep Q-Network (DDQN) Algorithmus entwickelt. Diese Arbeit untersucht durch einen direkten Vergleich in einer eigens entwickelten, merkmalsbasierten 2D-Rennspielumgebung, ob DDQN auch in einem kontrollierten Setting eine messbar überlegene Performanz erzielt. Der Ansatz isoliert den reinen algorithmischen Effekt, frei von den Komplexitäten des Lernens aus Bilddaten. Die empirischen Ergebnisse bestätigen, dass DDQN den Bias reduziert und eine im Mittel stabilere Lernleistung aufweist. Die Analyse zeigt jedoch, dass dieser Vorteil in der gewählten deterministischen Umgebung mit kleinem Aktionsraum von subtiler Natur ist. Die Arbeit demonstriert somit nicht nur die kontextabhängige Wirksamkeit von DDQN, sondern leistet auch einen Beitrag zur Methodik zugänglicher und reproduzierbarer Forschung im Reinforcement Learning.

Abstract

Compensation of overestimation bias in reinforcement learning: A comparison of DQN and DDQN in a racing game environment

The “Overestimation Bias” is a well-known shortcoming of the Deep Q-Network (DQN) algorithm that can lead to unstable training. The Double Deep Q-Network (DDQN) algorithm was developed as a solution. This thesis investigates whether DDQN achieves a measurably superior performance in a controlled setting by directly comparing it with DQN in a custom-built, feature-based 2D racing game environment. This approach isolates the pure algorithmic effect, free from the complexities of learning from image data. The empirical results confirm that DDQN reduces the bias and exhibits a more stable learning performance on average. However, the analysis shows that this advantage is subtle in the chosen deterministic environment with its small action space. Thus, the thesis not only demonstrates the context-dependent effectiveness of DDQN but also contributes to the methodology of accessible and reproducible research in Reinforcement Learning.

Inhaltsverzeichnis

1	Einleitung	1
1.1	Motivation	1
1.2	Zielsetzung	2
1.3	Aufbau der Arbeit	3
2	Grundlagen	4
2.1	Reinforcement Learning	4
2.1.1	Schlüsselkomponenten	4
2.1.2	Markov-Entscheidungsprozess	6
2.1.3	Wertefunktionen und Bellman-Gleichungen	7
2.1.4	Exploration vs. Exploitation	9
2.1.5	Kategorien von Reinforcement Learning-Algorithmen	11
2.2	Deep Q-Networks	12
2.2.1	Von der Tabelle zum neuronalen Netz	12
2.2.2	Stabilisierung des Lernprozesses	13
2.2.3	Das Problem der Wertüberschätzung	15
2.3	Double Deep Q-Networks	15
2.4	Verwendete Technologien	17
2.4.1	Python als Programmiersprache	17
2.4.2	Pygame zur Erstellung der Spielumgebung	17
2.4.3	Figma und die prozedurale Generierung der Streckengeometrie	18
2.4.4	TensorFlow und Keras für Deep Learning	19
2.4.5	NumPy für numerische Operationen	19
2.4.6	Visualisierung der Experimente mit TensorBoard und Plotly	20
2.4.7	Git und GitHub für Versionierung und Veröffentlichung	20
3	Verwandte Arbeiten	21
3.1	Reinforcement Learning in Simulationsumgebungen	21
3.1.1	End-to-End-Ansätze versus kompakte Zustandsrepräsentationen	21
3.1.2	Realismus und Komplexität von Simulationsumgebungen	21
3.2	Empirische Vergleiche von DQN und DDQN	22

3.3	Studien zur Mitigation des Overestimation Bias	23
3.3.1	Dueling-Netzwerkarchitekturen als Alternative	23
3.3.2	Orthogonale Ansätze zur Lernstabilität und Effizienz	23
3.3.3	Kombinatorische Ansätze und Abgrenzung	24
3.3.4	Abgrenzung und methodischer Beitrag der Arbeit	25
4	Problemstellung und Anforderungen	27
4.1	Problemstellung	27
4.1.1	Das Kernproblem: Der Overestimation Bias in DQN	27
4.1.2	Der postulierte Lösungsansatz: Double Q-Learning	28
4.1.3	Forschungslücke und Hypothesen	28
4.2	Anforderungen	29
4.2.1	Funktionale Anforderungen	29
4.2.2	Nicht-funktionale Anforderungen	31
5	Konzept und Methodik	33
5.1	Formalisierung der Rennspielumgebung	33
5.1.1	Zustandsraum ($\mathcal{S}$)	33
5.1.2	Aktionsraum ($\mathcal{A}$)	36
5.1.3	Belohnungsfunktion (R)	36
5.1.4	Episoden-Terminierung	37
5.2	Experimentelles Design	38
5.2.1	Netzwerkarchitektur und Hyperparameter	38
5.2.2	Statistische Absicherung	43
5.3	Evaluationsmetriken	45
6	Implementierung	46
6.1	Systemarchitektur im Überblick	46
6.2	Implementierung der Fahrdynamik und Sensorik	46
6.2.1	Simulation des Drift-Verhaltens	47
6.2.2	Implementierung der Distanzsensoren	47
6.3	Implementierung der Lernalgorithmen DQN und DDQN	47
6.3.1	DQN-Zielwertberechnung	48
6.3.2	DDQN-Zielwertberechnung	48
6.4	Steuerung des experimentellen Ablaufs	49

7	Experimente und Evaluation	50
7.1	Analyse der Lernperformanz	50
7.1.1	Quantitative Analyse des Lernverhaltens	50
7.1.2	Qualitative Analyse des Lernverhaltens	51
7.2	Quantifizierung des Overestimation Bias	52
8	Diskussion	54
8.1	Interpretation und Einordnung der Ergebnisse	54
8.2	Methodologischer Beitrag und Positionierung	55
8.3	Limitationen und Ausblick	56
9	Fazit	57
	Abkürzungsverzeichnis	59
	Glossar	60
	Abbildungsverzeichnis	63
	Tabellenverzeichnis	64
	Symbolverzeichnis	65
	Quellcodeverzeichnis	66
	Literatur	67

Kapitel 1

Einleitung

1.1 Motivation

Mit weitaus mehr möglichen Stellungen als Atomen im bekannten Universum galt das Brettspiel Go lange als die ultimative Herausforderung für künstliche Intelligenz. Anders als Schach, dessen Komplexität durch massive Rechenleistung beherrschbar wurde, widerstand sich Go der reinen Brute-Force-Analyse. Sein gewaltiger Suchraum und die Notwendigkeit für eine tiefgreifende, intuitive Mustererkennung machten es zu einem Bollwerk menschlicher Kreativität, das für Maschinen als unbezwingbar galt [1].

Diese Annahme wurde im März 2016 auf dramatische Weise widerlegt. DeepMinds Programm AlphaGo besiegte den Weltmeister Lee Sedol in einer Serie von Partien, die das Feld der KI für immer verändern sollten [1]. Der entscheidende Moment manifestierte sich in einem Zug, den der Go-Meister und Kommentator Fan Hui fassungslos mit den Worten kommentierte: „*It's not a human move. I've never seen a human play this move.*“ [2]. AlphaGo hatte nicht nur gewonnen; es hatte durch millionenfache Partien gegen sich selbst Strategien entwickelt, die über die etablierten menschlichen Denkmuster hinausgingen.

Dieser Erfolg basiert auf dem Prinzip des Reinforcement Learning (RL), einem Bereich des maschinellen Lernens, dessen technologische Grundlage durch das Deep Reinforcement Learning (DRL) revolutioniert wurde. Ein fundamentaler Wendepunkt, auf dessen Prinzipien auch AlphaGo aufbaute, war hierbei die Einführung des Deep Q-Network (DQN) Algorithmus [3]. Erstmals war es gelungen, einen einzigen, universellen Agenten zu schaffen, der direkt aus rohen Pixeldaten lernen konnte, eine Vielzahl von Atari-2600-Spielen auf übermenschlichem Niveau zu spielen. Durch diese wegweisende Arbeit etablierte sich die Atari-Umgebung als de-facto Standard-Benchmark in der RL-Forschung, auf dem unzählige nachfolgende Algorithmen entwickelt und verglichen wurden.

1.2 Zielsetzung

Die Stärke des pixelbasierten Ansatzes liegt in seiner Allgemeingültigkeit. Jedoch vermischt er methodisch zwei grundverschiedene Herausforderungen: das *Repräsentationslernen* (die Interpretation von Pixeldaten) und das *Strategielernen* (die Ableitung einer optimalen Handlungsstrategie). Verbesserungen könnten somit sowohl auf einem besseren Lernalgorithmus als auch auf einer robusteren Verarbeitung von visuellem Rauschen beruhen, was eine klare Attribution erschwert.

Zudem leidet der ursprüngliche DQN-Algorithmus unter einer subtilen, aber signifikanten Schwäche: dem sogenannten *Overestimation Bias* [4]. Aufgrund des Maximierungsoperators in seiner Update-Regel neigt DQN dazu, die wahren Werte möglicher Aktionen systematisch zu überschätzen. Diese übermäßig optimistische Einschätzung kann den Agenten zu riskanten oder suboptimalen Strategien verleiten und so zu instabilem Training und einer schlechteren finalen Performanz führen.

Als direkte Antwort auf dieses Problem wurde der Double Deep Q-Network (DDQN) Algorithmus entwickelt [5]. Durch eine clevere Entkopplung der Aktionsauswahl von der Aktionsbewertung reduziert DDQN diesen Bias nachweislich. Um den reinen algorithmischen Effekt dieser Verbesserung zu isolieren, frei von der Komplexität des Repräsentationslernens, verfolgt die vorliegende Arbeit einen *reduktionistischen Ansatz*. Anstatt auf hochdimensionale Pixeldaten zurückzugreifen, wird eine schlanke, merkmalsbasierte 2D-Rennspielumgebung entwickelt. Dieser methodische Fokus auf niedrigdimensionale, aber aussagekräftige Zustandsvektoren ist in der Forschung etabliert, um Kernmechanismen von RL-Algorithmen zu untersuchen, beispielsweise in klassischen Kontrollproblemen oder in der mobilen Robotik [6].

Dieser Ansatz führt direkt zur zentralen Forschungsfrage dieser Thesis:

Führt die durch DDQN implementierte Kompensation des Overestimation Bias auch in einer merkmalsbasierten Rennspielumgebung zu einer messbar überlegenen und stabileren Lern-Performanz im Vergleich zu Standard-DQN?

Zur Beantwortung dieser Frage wird untersucht, ob der DDQN-Agent, wie in der Theorie postuliert, systematisch realistischere Q-Wert-Schätzungen aufweist und ob sich diese genauere Einschätzung in einer überlegenen Fahrstrategie mit höheren Belohnungen manifestiert.

1.3 Aufbau der Arbeit

Die Struktur dieser Arbeit folgt einem logischen Aufbau, um die Forschungsfrage systematisch zu beantworten. [Kapitel 2](#) legt die theoretischen Grundlagen des Reinforcement Learning und stellt die Algorithmen DQN und DDQN detailliert vor. [Kapitel 3](#) ordnet die Arbeit durch die Betrachtung verwandter Studien in den wissenschaftlichen Kontext ein. [Kapitel 4](#) präzisiert die Problemstellung, leitet die Forschungshypothesen ab und definiert die Anforderungen an das experimentelle System. [Kapitel 5](#) beschreibt das methodische Vorgehen, einschließlich der Formalisierung der Rennspielumgebung und des detaillierten experimentellen Designs. [Kapitel 6](#) gibt Einblicke in die wesentlichen Aspekte der technischen Implementierung. [Kapitel 7](#) präsentiert die empirischen Ergebnisse des Vergleichs zwischen DQN und DDQN und evaluiert die aufgestellten Hypothesen. [Kapitel 8](#) interpretiert und diskutiert diese Ergebnisse, beleuchtet die Limitationen der Arbeit und ordnet die Erkenntnisse in einen breiteren Forschungskontext ein. Abschließend fasst [Kapitel 9](#) die Arbeit zusammen und gibt einen Ausblick auf mögliche zukünftige Forschungsrichtungen.

Kapitel 2

Grundlagen

Dieses Kapitel legt die theoretischen Grundlagen dar, die notwendig sind, um das in dieser Arbeit behandelte Themengebiet zu verstehen. Erst erfolgt die Einführung des Paradigmas des RL, dann werden die eingesetzten Algorithmen DQN und DDQN genau untersucht. Zum Abschluss erfolgt eine kurze Vorstellung der verwendeten Softwaretechnologien. Für eine konsistente Darstellung wird im Folgenden durchgängig die Notation gemäß Sutton & Barto [7] verwendet.

2.1 Reinforcement Learning

Reinforcement Learning, auf Deutsch bestärkendes oder verstärkendes Lernen, ist ein Teilbereich des maschinellen Lernens, der sich mit der Frage befasst, wie ein lernfähiger Agent durch Erfahrung optimale Strategien entwickelt, um in oft dynamischen oder unvorhergesehenen Umgebungen langfristige Ziele zu verfolgen [7]. Ein RL-Agent erlernt seine Fähigkeiten durch Interaktion mit seiner Umgebung, wobei er Rückmeldungen in Form von Belohnungen (Rewards) oder Bestrafungen erhält [8]. Dieses Vorgehen unterscheidet sich von den anderen Teilgebieten, dem überwachten Lernen, bei welchem dem Agenten korrekte Eingabe-Ausgabe-Paare gezeigt werden, dem unüberwachten Lernen, das Muster in ungelabelten Daten identifiziert [9, 10] sowie dem semi-überwachten Lernen, das Elemente beider Ansätze kombiniert [11, 12].

2.1.1 Schlüsselkomponenten

Der RL-Prozess ist grundlegend durch die Wechselwirkung zwischen Agent und Umgebung gekennzeichnet (siehe [Abbildung 2.1](#)) und kann laut Sutton und Barto [7] sowie Kaelbling et al. [13] in folgende Schlüsselkomponenten unterteilt werden:

- **Agent:** Die lernende Entität, die in einer Umgebung Entscheidungen trifft, um ein definiertes Ziel zu erreichen. In dieser Arbeit ist der Agent das computergesteuerte Fahrzeug, das lernt, eine Rennstrecke zu befahren.
- **Umgebung (Environment):** Die Welt, mit der der Agent interagiert. Für diese Arbeit stellt das in Pygame entwickelte Rennspiel die Umgebung dar, inklusive der Strecke und der Spielphysik.
- **Zustand (State, s):** Eine Repräsentation der aktuellen Situation, in der sich der Agent befindet. Ein Zustand ist definiert als eine Zusammenfassung aller relevanten Informationen, die ein Agent benötigt, um eine Entscheidung zu treffen. Im Kontext des Rennspiels ist beispielsweise die Position des Autos, welche durch den Abstand zu den Streckenrändern in verschiedenen Winkeln bestimmt wird, ein Teilaspekt des Zustandes. Die Menge aller möglichen Zustände wird als Zustandsraum S bezeichnet.
- **Aktion (Action, a):** Eine Entscheidung, die der Agent in einem bestimmten Zustand treffen kann. Im Rennspiel kann der Agent z.B. entscheiden, ob er beschleunigen, bremsen oder die Richtung ändern soll. Die Menge aller möglichen Aktionen in einem Zustand s wird als Aktionsraum $A(s)$ bezeichnet.
- **Belohnung (Reward, r):** Ein numerisches Feedbacksignal, das den Erfolg der Aktion bewertet. Die Belohnung signalisiert, wie gut oder schlecht die gewählte Aktion im Hinblick auf das übergeordnete Ziel war. In dieser Arbeit erhält der Agent eine positive Belohnung, wenn er Checkpoints auf der Strecke erreicht, und eine negative Belohnung, wenn er mit der Streckenwand kollidiert.
- **Strategie (Policy, π):** Die Strategie oder Verhaltensweise des Agenten, die bestimmt, welche Aktion a der Agent in einem Zustand s wählt. Es wird unterschieden zwischen:
 - **Deterministisch ($\pi(s)$):** In einem fest definierten Zustand s wird stets die identische Aktion ausgewählt.
 - **Stochastisch ($\pi(a \mid s)$):** Die Entscheidung erfolgt probabilistisch, d. h., die Wahl des Agenten für die Aktion a hängt mit einer gewissen Wahrscheinlichkeit vom aktuellen Zustand s ab.

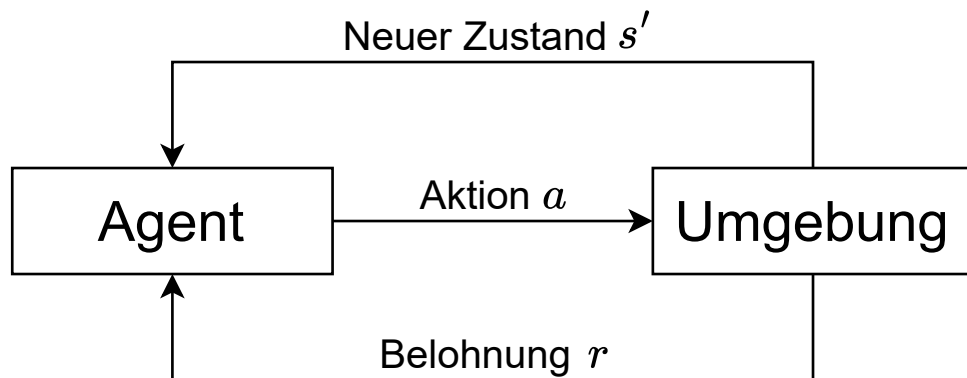


Abbildung 2.1: Der Agent interagiert durch die Aktion a mit der Umgebung und erhält daraus den neuen Zustand s' und die Belohnung r . Diese Interaktion bildet den Kern des Reinforcement-Learning-Prozesses.

2.1.2 Markov-Entscheidungsprozess

Nachdem im vorherigen Abschnitt der grundlegende Interaktionszyklus zwischen Agent und Umgebung sowie dessen Schlüsselkomponenten erläutert wurden, wird nun der formale Rahmen vorgestellt, der es ermöglicht, solche sequentiellen Entscheidungsprobleme präzise zu modellieren: der Markov-Entscheidungsprozess (Markov Decision Process (MDP)). Viele komplexe Probleme im RL, insbesondere solche, bei denen ein Agent durch eine Abfolge von Entscheidungen ein langfristiges, kumulatives Ziel maximieren soll, lassen sich effektiv durch MDPs darstellen [7]. Indem ein MDP die Dynamik der Umgebung, die Handlungsoptionen und Belohnungen formal strukturiert, schafft er die theoretische Grundlage für die Entwicklung, Analyse und Anwendung nahezu aller gängigen Reinforcement-Learning-Algorithmen [13]. In *Artificial Intelligence: A Modern Approach* von Russell und Norvig [14] werden die grundlegenden Elemente eines MDP eingeführt:

- $\mathcal{S}$: eine endliche Menge von Zuständen (State Space).
- $\mathcal{A}(s)$: eine endliche Menge von Aktionen, die im Zustand s verfügbar sind (Action Space).
- $p(s' \mid s, a)$: die Zustandsübergangswahrscheinlichkeit, also die Wahrscheinlichkeit, vom Zustand s durch Ausführung der Aktion a in den Folgezustand s' überzugehen.

- $r(s, a, s')$: die Belohnungsfunktion, die den erwarteten unmittelbaren Ertrag beschreibt, wenn der Agent im Zustand s die Aktion a wählt und in den Zustand s' gelangt.

Ein zentraler Bestandteil des MDP-Formalismus ist die Markov-Eigenschaft. Diese besagt, dass die Wahrscheinlichkeitsverteilung der Zufallsvariablen S_{t+1} und R_{t+1} ausschließlich vom aktuellen Zustand S_t und der gewählten Aktion A_t abhängen – und nicht von der vorherigen Geschichte der Zustände und Aktionen [8]. Diese Eigenschaft vereinfacht die Analyse und die Lösung vieler RL-Probleme erheblich. In realitätsnahen Szenarien ist die Markov-Eigenschaft nicht immer strikt erfüllt, insbesondere wenn der beobachtete Zustand nicht alle relevanten Umweltinformationen vollständig erfasst [15]. Dennoch wird sie als nützliche Approximation verwendet und bildet die Grundlage für die Anwendung vieler RL-Algorithmen, einschließlich Q-Learning und DQN [13, 15].

2.1.3 Wertefunktionen und Bellman-Gleichungen

Nachdem das Problem nun formal als MDP beschrieben werden kann, besteht die nächste Herausforderung darin, eine optimale Strategie für den Agenten innerhalb dieses Frameworks zu finden. Ein zentrales Konzept hierfür sind Wertefunktionen, die den erwarteten langfristigen Nutzen (Return) quantifizieren, den ein Agent von einem bestimmten Zustand oder von der Ausführung einer bestimmten Aktion in einem bestimmten Zustand erwarten kann, wenn er einer spezifischen Strategie (Policy) π folgt [7]. Diese Wertefunktionen sind entscheidend, da sie es ermöglichen, die „Güte“ von Zuständen oder Zustand-Aktions-Paaren zu bewerten und somit als Grundlage für die Verbesserung von Strategien dienen [16].

Da die Summe der zukünftigen Belohnungen in episodischen Aufgaben zwar endlich, in kontinuierlichen Aufgaben (oder solchen mit sehr langen Episoden) jedoch potenziell unendlich sein kann, wird ein Diskontierungsfaktor (Discount Factor) $\gamma \in [0, 1]$ eingeführt [7]. Dieser Faktor erfüllt zwei wesentliche Zwecke. Einerseits sorgt er für die *mathematische Konvergenz*: Für $\gamma < 1$ stellt er sicher, dass die Summe der diskontierten Belohnungen auch bei einem unendlichen Zeithorizont konvergiert, sofern die Einzelbelohnungen beschränkt sind [17]. Andererseits modelliert er eine *Präferenz für frühere Belohnungen*, da ein Wert von $\gamma < 1$ unmittelbare Belohnungen stärker gewichtet als spätere. Dies kann Unsicherheit über die ferne Zukunft abbilden oder eine Präferenz für kurzfristige Erfolge darstellen [7]. Im Extremfall $\gamma = 0$ betrachtet der Agent nur die unmittelbare Belohnung, während bei $\gamma = 1$ alle zukünftigen Belohnungen gleich gewichtet werden, was in nicht-episodischen Aufgaben zu Konvergenzproblemen führen kann.

Die kumulierte diskontierte Belohnung ab Zeitpunkt t , auch als Return G_t bezeichnet, ist definiert als:

$$G_t \doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \quad (2.1)$$

wobei R_{t+k+1} die Belohnung ist, die zum Zeitschritt $t + k + 1$ erhalten wird, nachdem in Zustand S_{t+k} die Aktion A_{t+k} ausgeführt wurde [7].

Es wird hauptsächlich zwischen zwei Arten von Wertefunktionen unterschieden. Die erste ist die *Zustands-Wertefunktion (State-Value Function)*, $v_{\pi}(s)$. Sie gibt den erwarteten Return an, wenn der Agent im Zustand s startet und danach der Policy π folgt. Der Erwartungswert $\mathbb{E}_{\pi}[\cdot]$ wird hierbei über alle möglichen Trajektorien genommen, die sich aus der Policy π ergeben [7]. Die Funktion drückt also aus, wie „gut“ es ist, sich in einem bestimmten Zustand zu befinden. Im Kontext des Rennspiels hätte ein Zustand kurz vor einem Checkpoint einen hohen Wert, während ein Zustand, der auf eine Wand zusteuert, einen niedrigen Wert hätte. Ein Nachteil ist jedoch, dass die State-Value-Funktion allein keine direkte Handlungsanweisung liefert.

Die zweite Art ist die *Aktions-Wertefunktion (Action-Value Function)*, $q_{\pi}(s, a)$. Sie gibt den erwarteten Return an, wenn der Agent im Zustand s startet, die Aktion a ausführt und danach der Policy π folgt [7]. Hierin liegt ihr entscheidender Vorteil für modellfreie Agenten: Die Q-Funktion bewertet direkt die Handlungsoptionen. Im Rennspiel hätte beispielsweise die Aktion „beschleunigen“ einen hohen Q-Wert, wenn sich das Auto mittig auf einer Geraden befindet, aber einen extrem niedrigen Q-Wert, wenn es auf eine Wand zusteuert. Diese Q-Werte (Quality-Werte) sind besonders nützlich für die Aktionsauswahl, da der Agent in einem Zustand s einfach die Aktion a wählen kann, die $q_{\pi}(s, a)$ maximiert, um gierig zu handeln [3].

Wertefunktionen erfüllen eine rekursive Beziehung, die als Bellman-Gleichung bekannt ist. Diese Gleichungen zerlegen den Wert eines Zustands (oder Zustand-Aktions-Paars) in die unmittelbare Belohnung plus den diskontierten Wert des Folgezustands (oder der Folgezustand-Aktions-Paare) [18]. Sie sind fundamental für viele RL-Algorithmen, da sie eine Basis für das Lernen und die Berechnung von Werten bieten.

Für die Zustands-Wertefunktion $v_{\pi}(s)$ lautet die Bellman-Gleichung:

$$v_{\pi}(s) \doteq \sum_a \pi(a|s) \sum_{s', r} p(s', r|s, a) [r + \gamma v_{\pi}(s')] \quad (2.2)$$

für alle $s \in \mathcal{S}$ [7]. Hierbei ist $\pi(a|s)$ die Wahrscheinlichkeit, Aktion a im Zustand s unter Policy π zu wählen, und $p(s', r|s, a)$ ist die Wahrscheinlichkeit, in Zustand s' mit Belohnung r zu landen, nachdem Aktion a im Zustand s ausgeführt wurde.

Für die Aktions-Wertefunktion $q_\pi(s, a)$ lautet die Bellman-Gleichung:

$$q_\pi(s, a) \doteq \sum_{s', r} p(s', r | s, a) [r + \gamma \sum_{a'} \pi(a' | s') q_\pi(s', a')] \quad (2.3)$$

für alle $s \in \mathcal{S}, a \in \mathcal{A}(s)$ [7].

Das Ziel in RL ist es oft, die optimale Policy π^* zu finden, d.h., die Policy, die zu höheren oder gleich hohen erwarteten Returns führt als jede andere Policy für alle Zustände [7]. Es existieren immer optimale Policies, und sie alle teilen sich dieselbe optimale Zustands-Wertefunktion $v_*(s) \doteq \max_\pi v_\pi(s)$ und zusätzlich dieselbe optimale Aktions-Wertefunktion $q_*(s, a) \doteq \max_\pi q_\pi(s, a)$.

Die Bellman-Gleichungen für die optimalen Wertefunktionen, bekannt als Bellman-Optimalitätsgleichungen, sind:

$$v^*(s) = \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma v_*(s')] \quad (2.4)$$

und

$$q^*(s, a) = \sum_{s', r} p(s', r | s, a) [r + \gamma \max_{a'} q_*(s', a')] \quad (2.5)$$

[7]. Die Bellman-Optimalitätsgleichung für $q_*(s, a)$ ist die Grundlage für viele Lernalgorithmen, einschließlich Q-Learning, da sie direkt angibt, wie der Q-Wert einer optimalen Aktion mit den optimalen Q-Werten der nachfolgenden Aktionen zusammenhängt.

2.1.4 Exploration vs. Exploitation

Das Exploration-Exploitation-Dilemma ist ein grundlegendes Problem in Entscheidungsprozessen, das besonders im RL von zentraler Bedeutung ist [19]. Es beschreibt den Zielkonflikt, vor dem ein Agent steht: Soll er sein aktuelles Wissen nutzen, um die bestmögliche Aktion basierend auf den bisherigen Erfahrungen auszuführen (Exploitation), oder soll er neue, potenziell unbekannte Aktionen oder Zustände erkunden, die langfristig zu besseren Ergebnissen führen könnten (Exploration) [20]. Dieser Konflikt entsteht, da der Agent zu Beginn keine vollständige Kenntnis der Umgebungsdynamik und der Belohnungsstruktur besitzt. Um optimale Strategien zu entdecken, ist es unerlässlich, Informationen durch Exploration zu sammeln, was jedoch oft auf Kosten unmittelbarer Belohnungen geht [17, 21].

Eine ausgewogene Balance zwischen Exploration und Exploitation ist für einen erfolgreichen Lernprozess unerlässlich. Eine reine Exploitation-Strategie birgt das Risiko, dass der Agent in einem suboptimalen lokalen Optimum verharrt und niemals bessere Verhaltens-

weisen entdeckt [22]. Umgekehrt kann übermäßige Exploration zu ineffizientem Lernen führen, da der Agent zu viel Zeit mit dem Ausprobieren zufälliger Aktionen verbringt, ohne das bereits erworbene Wissen effektiv zu nutzen [23]. Weiterhin ist die Balance zwischen Exploration und Exploitation ein dynamischer Prozess, da eine feste Explorationsrate nachweislich zu suboptimalem, linearem Bedauern („Regret“) führt. Erfolgreiche Strategien passen sich daher an: Sie reduzieren die Exploration systematisch mit zunehmendem Wissen, um von der anfänglichen Entdeckung zur späteren, effizienten Ausbeutung der besten Optionen überzugehen [24].

Zur Bewältigung dieses Dilemmas werden diverse Strategien angewendet. Die wohl bekannteste Methode ist das ε -greedy-Verfahren, bei dem der Agent mit einer geringen Wahrscheinlichkeit ε eine zufällige Aktion wählt, anstatt der besten bekannten. Im Rennspiel erlaubt dies dem Agenten, gelegentlich von der optimalen Linie abzuweichen, um beispielsweise neue Kurvenradien zu testen. Typischerweise wird ε im Laufe des Trainings reduziert, um den Fokus von der Exploration zur Exploitation zu verlagern [7]. Anspruchsvollere Methoden wie Upper Confidence Bound gehen gezielter vor, indem sie Aktionen bevorzugen, deren Wertschätzung noch mit hoher Unsicherheit behaftet ist, und somit die Erkundung gezielt in vielversprechende, aber wenig erforschte Bereiche lenken [25].

Die Effektivität dieser Strategien hängt jedoch entscheidend von der Belohnungsstruktur der Umgebung ab. Eine zentrale Herausforderung sind spärliche und trügerische Belohnungen (sparse and deceptive rewards) [26]. Wenn ein Agent beispielsweise nur am Ende einer erfolgreichen Runde eine positive Belohnung erhält (spärlich), ist es für einfache Explorationsmethoden wie ε -greedy extrem schwierig, die lange Kette korrekter Aktionen zufällig zu entdecken, die zu diesem Erfolg geführt hat. Dies motiviert fortgeschrittene Ansätze wie die intrinsische Motivation, bei der Neugier und die Entdeckung neuer Zustände selbst belohnt werden [27]. Ein solcher Explorationsbonus schafft ein dichteres Belohnungssignal, das den Agenten dazu anleitet, aktiv die Strecke zu erkunden, anstatt auf seltene externe Belohnungen zu warten.

Diese Herausforderungen verdeutlichen, dass das Design der Belohnungsfunktion, auch als Reward Shaping bekannt, ein zentraler Bestandteil der Lösungsstrategie ist. Eine gut konzipierte Belohnungsfunktion – etwa durch das Belohnen von Checkpoint-Passagen – kann die Explorationsprobleme signifikant entschärfen. Sie leitet den Agenten effektiv an und stellt sicher, dass das Fahrzeug nicht ziellos umherirrt (zu viel Exploration) oder sich vorschnell auf einer suboptimalen Route festfährt (zu viel Exploitation). Die Wahl der Explorationsstrategie und das Design des Belohnungssystems sind somit untrennbar miteinander verbunden, um ein robustes und effizientes Lernen zu ermöglichen.

2.1.5 Kategorien von Reinforcement Learning-Algorithmen

Nachdem die grundlegenden Konzepte wie der MDP und die Bellman-Gleichungen etabliert sind, ist es notwendig, die Vielfalt der Reinforcement-Learning-Algorithmen zu strukturieren. Diese lassen sich anhand mehrerer zentraler Kriterien klassifizieren, was nicht nur das Verständnis erleichtert, sondern auch bei der Auswahl eines geeigneten Algorithmus für ein spezifisches Problem hilft [7].

Eine erste grundlegende Unterscheidung erfolgt danach, ob der Agent ein explizites Modell der Umgebung erlernt oder nicht [13]. *Modellbasierte* Ansätze versuchen, die Umgebungsdynamik (d.h. Zustandsübergänge und Belohnungen) zu lernen, um durch interne Simulation und Planung eine optimale Strategie zu finden. Dies kann die Dateneffizienz drastisch erhöhen, birgt aber das Risiko, dass Fehler im gelernten Modell die Performanz limitieren [7]. Im Gegensatz dazu lernen *modellfreie* Algorithmen direkt aus der Interaktion mit der Umgebung. Die in dieser Arbeit untersuchten Algorithmen DQN und DDQN gehören zu dieser Kategorie, da sie auf ein explizites Erlernen der Spielphysik verzichten – ein Ansatz, der sich in Umgebungen mit schwer zu modellierender Dynamik als robuster und praktikabler erwiesen hat [3].

Innerhalb der modellfreien Algorithmen wird weiter danach unterschieden, was genau der Agent lernt. *Wertbasierte* Methoden fokussieren sich auf das Erlernen einer Wertefunktion, typischerweise der Aktions-Wertefunktion $Q(s, a)$. Die Strategie wird dabei implizit durch die Auswahl der Aktion mit dem höchsten geschätzten Wert abgeleitet. Im Gegensatz dazu lernen strategiebasierte Methoden, wie beispielsweise der REINFORCE-Algorithmus, eine Policy $\pi(a|s)$ direkt, was sie besonders für Probleme mit kontinuierlichen Aktionsräumen geeignet macht [28]. Eine hybride Form stellen *Actor-Critic*-Methoden dar, die beide Ansätze kombinieren. Die für diese Arbeit zentralen Algorithmen, DQN und DDQN, sind rein wertbasiert.

Die letzte entscheidende Klassifizierung betrifft die Art der verwendeten Trainingsdaten. *On-Policy*-Methoden, wie der klassische SARSA-Algorithmus, lernen ausschließlich aus Erfahrungen, die mit der aktuellen Policy gesammelt wurden. Dies erfordert nach jeder Policy-Verbesserung neue Daten. Demgegenüber sind *Off-Policy*-Methoden wie Q-Learning und dessen Derivate DQN und DDQN in der Lage, auch aus älteren, mitunter veralteten Erfahrungen zu lernen. Diese Eigenschaft ist die Voraussetzung für die Nutzung von *Experience Replay*, einer Technik, bei der vergangene Transitionen in einem Puffer gespeichert und wiederholt zum Training herangezogen werden. Dies erhöht die Dateneffizienz und ist fundamental, um den Lernprozess mit tiefen neuronalen Netzen zu stabilisieren [3].

Zusammenfassend lassen sich die Algorithmen DQN und DDQN als *modellfreie, wertbasierte und Off-Policy-Methoden* einordnen. Diese spezifische Kombination macht sie einerseits leistungsfähig für komplexe, modellfreie Aufgaben, bringt andererseits jedoch auch inhärente Herausforderungen mit sich, wie etwa die Tendenz zur Über- oder Unterschätzung von Q-Werten [4, 29].

2.2 Deep Q-Networks

Nachdem die Grundlagen des Reinforcement Learning gelegt wurden, widmet sich dieses Kapitel dem ersten zentralen Algorithmus dieser Arbeit: den Deep Q-Networks. DQN stellt einen Meilenstein in der Geschichte des RL dar, da es als erster Algorithmus in der Lage war, durch die Kombination von Q-Learning mit tiefen neuronalen Netzen menschenähnliche oder sogar übermenschliche Leistungen in komplexen Umgebungen wie den Atari-Spielen zu erzielen [3]. Dieser Durchbruch machte RL für eine breite Klasse von Problemen mit hochdimensionalen Zustandsräumen praktisch anwendbar. Im Folgenden wird zunächst der Übergang vom tabellarischen Q-Learning zur Funktionsapproximation mittels neuronaler Netze erläutert. Anschließend werden die beiden Schlüsseltechniken vorgestellt, die das Training erst stabil und effektiv machen: Experience Replay und feste Q-Ziele. Schließlich wird das inhärente Problem der Wertüberschätzung beleuchtet, das die Motivation für die Entwicklung von DDQN liefert.

2.2.1 Von der Tabelle zum neuronalen Netz

Der klassische Q-Learning-Algorithmus, wie von Watkins eingeführt [30], basiert auf der Speicherung der Q-Werte für jedes mögliche Zustand-Aktions-Paar in einer Tabelle, der sogenannten Q-Tabelle. Der Lernprozess aktualisiert iterativ die Einträge dieser Tabelle mithilfe der Bellman-Optimalitätsgleichung. Die Update-Regel für einen Übergang (s, a, r, s') lautet:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right] \quad (2.6)$$

wobei α die Lernrate ist [7]. Dieser tabellarische Ansatz ist jedoch nur für Probleme mit kleinen, diskreten Zustands- und Aktionsräumen, wie etwa ein Labyrinthspiel mit einer begrenzten Anzahl von Feldern [31], praktikabel. Sobald die Anzahl der Zustände sehr groß wird oder die Zustände kontinuierliche Werte annehmen – wie die genaue Position und Ausrichtung eines Fahrzeugs im Rennspiel – wird die Verwaltung einer solchen Tabelle beinahe unmöglich. Der Speicherbedarf explodiert, und die meisten Zustände würden während des Trainings nie oder nur selten besucht werden, was ein effektives Lernen

verhindert [7]. Hier setzt die Idee der Funktionsapproximation an. Anstatt jeden Q-Wert einzeln zu speichern, wird eine parametrisierte Funktion $Q(s, a; \theta)$ verwendet, die die Q-Werte für beliebige Zustände und Aktionen schätzt. Die Parameter θ dieser Funktion werden während des Trainings angepasst. Tiefe neuronale Netze haben sich als besonders leistungsfähige Funktionsapproximatoren erwiesen, da sie in der Lage sind, komplexe, nicht-lineare Zusammenhänge und relevante Merkmale aus hochdimensionalen Eingabedaten zu erlernen [32]. Ein Deep Q-Network ist somit ein neuronales Netz, das einen Zustand als Eingabe erhält und als Ausgabe die geschätzten Q-Werte für jede mögliche Aktion ausgibt. Für das in dieser Arbeit entwickelte Rennspiel würde das Netzwerk also einen Vektor, der unter anderem die Sensorabstände repräsentiert, als Input verarbeiten und einen Vektor mit Q-Werten für Aktionen wie „links“ oder „rechts“ als Output liefern. Die optimale Aktion in einem Zustand kann dann einfach durch Auswahl der Aktion mit dem höchsten vorhergesagten Q-Wert bestimmt werden: $a = \operatorname{argmax}_a Q(s, a; \theta)$. Der Lernprozess wird zu einem Problem des überwachten Lernens umformuliert. Das Ziel ist es, die Netzwerkparameter so anzupassen, dass der Unterschied zwischen der Vorhersage des Netzwerks und einem Zielwert (Target) minimiert wird. Als Verlustfunktion wird typischerweise der mittlere quadratische Fehler (Mean Squared Error, MSE) zwischen dem vorhergesagten Q-Wert und dem Q-Learning-Zielwert verwendet:

$$L(\theta) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} \left[\left(\underbrace{r + \gamma \max_{a'} Q(s', a'; \theta)}_{\text{Zielwert (Target)}} - \underbrace{Q(s, a; \theta)}_{\text{Vorhersage}} \right)^2 \right] \quad (2.7)$$

Dieser Verlust wird dann mittels Gradientenabstieg optimiert, um die Gewichte θ des Netzwerks zu aktualisieren [3].

2.2.2 Stabilisierung des Lernprozesses

Die direkte Anwendung von neuronalen Netzen auf Q-Learning, wie in Gleichung 2.7 beschrieben, führt jedoch zu erheblichen Instabilitäten im Lernprozess. Dies liegt an zwei Hauptproblemen: Erstens sind die aufeinanderfolgenden Trainingsdaten stark korreliert, da sie aus einer kontinuierlichen Trajektorie stammen. Zweitens ist das Ziel (Target), gegen das der Verlust berechnet wird, nicht stationär, da es vom selben Netzwerk abhängt, dessen Gewichte bei jedem Schritt aktualisiert werden [3]. Um diese Probleme zu lösen, führte das DeepMind-Team zwei entscheidende Techniken ein.

Experience Replay

Um die Korrelationen zwischen aufeinanderfolgenden Trainingssamples aufzubrechen, wird die Off-Policy-Natur von Q-Learning ausgenutzt. Anstatt das Netzwerk direkt nach jeder Interaktion zu trainieren, werden die Erfahrungen des Agenten, Tupel der Form $(s_t, a_t, r_{t+1}, s_{t+1})$, in einem großen Speicher mit fester Größe, dem sogenannten Replay Buffer (oder Replay Memory), abgelegt [33]. Für das Training des Netzwerks werden dann nicht die unmittelbar letzten Erfahrungen verwendet, sondern zufällig ausgewählte Mini-Batches von Übergängen aus diesem Puffer. Dieser Prozess bietet mehrere entscheidende Vorteile. Erstens führt das zufällige Sampeln zur *Reduzierung von Korrelationen*, da die starken zeitlichen Abhängigkeiten in den Trainingsdaten aufgebrochen werden. Dies reduziert die Varianz der Updates und erfüllt die Annahmen vieler Optimierungsalgorithmen, die von unabhängigen und identisch verteilten (i.i.d.) Daten ausgehen, besser [3]. Zweitens wird die *Dateneffizienz erhöht*, da jede Erfahrung mehrfach für Trainingsupdates wiederverwendet werden kann. Dies ist besonders im Kontext des Rennspiels wichtig, da seltene, aber entscheidende Ereignisse – wie das erfolgreiche Meistern einer schwierigen Kurve oder ein knapper Crash – so wiederholt zum Lernen herangezogen werden können. Drittens wird dem *Vergessen vorgebeugt*: Ohne einen Replay Buffer würde der Agent schnell verdrängen, was er in früheren Phasen des Trainings gelernt hat, während der Puffer sicherstellt, dass auch alte Erfahrungen im Lernprozess präsent bleiben.

Feste Q-Ziele durch Target-Netzwerke

Das zweite Problem ist, dass sich das Ziel in der Verlustfunktion (Gleichung 2.7) bei jedem Trainingsschritt ändert, da es von den gleichen Gewichten θ abhängt, die gerade aktualisiert werden. Dies ist vergleichbar mit dem Versuch, ein sich bewegendes Ziel zu treffen, und kann zu Oszillationen und Divergenz des Lernprozesses führen [3]. Die Lösung besteht in der Verwendung von zwei separaten neuronalen Netzen. Das erste ist das *Online-Netzwerk* ($Q(s, a; \theta)$), auch Policy-Netzwerk genannt, welches bei jedem Trainingsschritt aktualisiert und zur Auswahl der Aktionen während der Interaktion mit der Umgebung verwendet wird. Das zweite ist das *Target-Netzwerk* ($Q(s', a'; \theta^-)$). Hierbei handelt es sich um eine periodisch aktualisierte Kopie des Online-Netzwerks. Seine Gewichte θ^- werden für einen festen Zeitraum C eingefroren und dienen ausschließlich zur Berechnung des Zielwerts in der Verlustfunktion. Die Verlustfunktion wird dadurch modifiziert zu:

$$L(\theta) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} \left[\left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta) \right)^2 \right] \quad (2.8)$$

Alle C Schritte werden die Gewichte des Online-Netzwerks in das Target-Netzwerk kopiert: $\theta^- \leftarrow \theta$. Indem das Ziel für eine Weile stabil gehalten wird, wird der Lernprozess deutlich stabiler und konvergiert zuverlässiger [3]. Im Rennspiel-Szenario bedeutet dies, dass der Agent seine aktuellen Fahrstrategien anhand von leicht veralteten, aber stabilen Einschätzungen über die Zukunft bewertet, was sprunghafte Änderungen in der Fahrweise verhindert.

2.2.3 Das Problem der Wertüberschätzung

Trotz dieser Stabilisierungen leidet der DQN-Algorithmus unter einem subtilen, aber signifikanten Problem: einer systematischen Überschätzung der Q-Werte (Overestimation Bias) [4]. Dieses Problem entsteht durch den Maximierungsoperator in der Update-Regel (Gleichung 2.8). Da die Q-Werte nur Schätzungen sind und mit einem gewissen Fehler (Rauschen) behaftet sind, führt die max-Operation dazu, dass tendenziell Aktionen mit positiv fehlerbehafteten Schätzwerten bevorzugt werden. Wenn die Schätzungen für alle Aktionen im Folgezustand s' verrauscht sind, wird der Maximierungsoperator mit hoher Wahrscheinlichkeit einen Wert auswählen, der über dem wahren maximalen Q-Wert liegt. Diese positive Verzerrung propagiert sich durch die Bellman-Updates rückwärts durch das System und kann zu einer erheblichen Überschätzung der wahren Werte führen [5]. Für das Rennspiel könnte dies bedeuten, dass der Agent übermäßig optimistische Schätzungen für Aktionen entwickelt, die ihn nahe an die Wand führen. Obwohl diese Aktionen im Mittel gefährlich sind, könnte eine zufällig hohe Q-Wert-Schätzung den Agenten dazu verleiten, diese riskanten Manöver zu wählen, was zu suboptimalem Fahrverhalten und häufigeren Kollisionen mit der Streckenbegrenzung führen kann. Die Reduzierung dieser Überschätzung ist das Hauptziel des DDQN-Algorithmus, der im folgenden Kapitel vorgestellt wird.

2.3 Double Deep Q-Networks

Wie im vorherigen Abschnitt dargelegt, neigt der DQN-Algorithmus zu einer systematischen Überschätzung der Q-Werte. Dieses als Overestimation Bias bekannte Problem kann zu suboptimalen Strategien führen, da der Agent möglicherweise Aktionen bevorzugt, deren Wert nur durch Schätzfehler künstlich erhöht wurde. Um diesem Problem entgegenzuwirken, stellten van Hasselt, Guez und Silver die Idee des Double Q-Learning in den Kontext tiefer neuronaler Netze und entwickelten den Double Deep Q-Network Algorithmus [5].

Die Kernidee des DDQN besteht darin, die Auswahl der besten Aktion im Folgezustand von der Bewertung dieser Aktion zu entkoppeln. Im Standard-DQN werden beide Schritte vom selben Netzwerk – dem Target-Netzwerk – durchgeführt. Der max-Operator wählt die Aktion mit dem höchsten Q-Wert und verwendet gleichzeitig den Q-Wert dieser Aktion zur Berechnung des Ziels (siehe Gleichung 2.8). DDQN bricht diesen Prozess auf, indem es die beiden bereits vorhandenen Netzwerke, das Online-Netzwerk und das Target-Netzwerk, für unterschiedliche Aufgaben einsetzt:

1. **Aktionsauswahl:** Das Online-Netzwerk mit den aktuellen Gewichten θ wird verwendet, um diejenige Aktion a' im Folgezustand s' zu identifizieren, die es für die beste hält. Dies geschieht durch die Berechnung von $\operatorname{argmax}_{a'} Q(s', a'; \theta)$.
2. **Aktionsbewertung:** Der Q-Wert dieser ausgewählten Aktion wird jedoch nicht vom Online-Netzwerk selbst abgelesen, sondern vom stabilen Target-Netzwerk mit den älteren Gewichten θ^- .

Durch diese Entkopplung wird die Wahrscheinlichkeit verringert, dass eine durch Zufall zu hoch geschätzte Aktion auch zur Berechnung des Zielwerts herangezogen wird. Selbst wenn das Online-Netzwerk eine Aktion aufgrund eines Schätzfehlers fälschlicherweise als optimal einstuft, ist es unwahrscheinlich, dass das unabhängige Target-Netzwerk für dieselbe Aktion ebenfalls einen stark überschätzten Wert besitzt [5].

Formal wird dazu die Zielwertberechnung in der Verlustfunktion modifiziert. Anstelle des einfachen Maximums über die Q-Werte des Target-Netzwerks lautet das Ziel y_t für den DDQN Algorithmus:

$$y_t \doteq r + \gamma Q(s', \operatorname{argmax}_{a'} Q(s', a'; \theta), \theta^-) \quad (2.9)$$

Die Verlustfunktion bleibt ansonsten identisch, verwendet aber dieses neue, potenziell weniger verzerrte Ziel:

$$L(\theta) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} [(y_t - Q(s, a; \theta))^2] \quad (2.10)$$

Diese auf den ersten Blick kleine Änderung an der Update-Regel hat sich in der Praxis als sehr wirkungsvoll erwiesen. Sie führt zu konservativeren und realistischeren Q-Wert-Schätzungen, was wiederum stabilere Lernprozesse und oft eine bessere finale Performanz des Agenten zur Folge hat [5]. Im Kontext des Rennspiels soll diese Modifikation den Agenten davor bewahren, übermäßig riskante Fahrmanöver zu wählen, deren potenzieller Erfolg vom Standard-DQN überschätzt werden könnte.

2.4 Verwendete Technologien

Die Umsetzung des in dieser Arbeit beschriebenen Projekts – von der Erstellung der Simulationsumgebung bis zum Training und der Evaluation der Reinforcement-Learning-Agenten – erfordert eine Reihe von spezialisierten Softwaretechnologien. Die Auswahl des richtigen Technologie-Stacks ist entscheidend für eine effiziente Entwicklung, die Durchführung reproduzierbarer Experimente und die aussagekräftige Visualisierung der Ergebnisse. Dieses Kapitel stellt die wesentlichen Softwarekomponenten vor, die in dieser Arbeit zum Einsatz kamen, und begründet deren jeweilige Auswahl.

2.4.1 Python als Programmiersprache

Als primäre Programmiersprache für das gesamte Projekt wurde Python in der Version 3.10.17 gewählt. Diese Entscheidung ist nicht nur auf die allgemeine Popularität von Python im Bereich des maschinellen Lernens zurückzuführen, sondern gründet sich auf konkrete Synergien, die für die Umsetzung dieser Arbeit entscheidend waren. Insbesondere die nahtlose Integration des wissenschaftlichen Computing-Stacks bestehend aus *NumPy* für die effiziente Verwaltung des Replay Buffers und der Zustandsvektoren, *TensorFlow/Keras* für die Implementierung der DQN- und DDQN-Agenten und *Pygame* für die Erstellung der Simulationsumgebung wäre in einer anderen Sprache nur mit erheblichem Mehraufwand möglich gewesen.

2.4.2 Pygame zur Erstellung der Spielumgebung

Die Rennspielumgebung, die als Testfeld für die RL-Agenten dient, wurde mit Pygame entwickelt. Pygame ist eine Sammlung von Python-Modulen, die auf der Simple DirectMedia Layer (SDL) Bibliothek aufbaut und zur Erstellung von Videospielen und Multimedia-Anwendungen dient. Für dieses Projekt war Pygame die ideale Wahl, da es im Gegensatz zu großen, komplexen Spiel-Engines wie Unity oder Unreal eine leichtgewichtige und flexible Alternative darstellt. Es gewährt die volle Kontrolle über die Gameloop, die Physiksimulation und die Zustandsrepräsentation, was für die Entwicklung einer benutzerdefinierten RL-Umgebung unerlässlich ist.

2.4.3 Figma und die prozedurale Generierung der Streckengeometrie

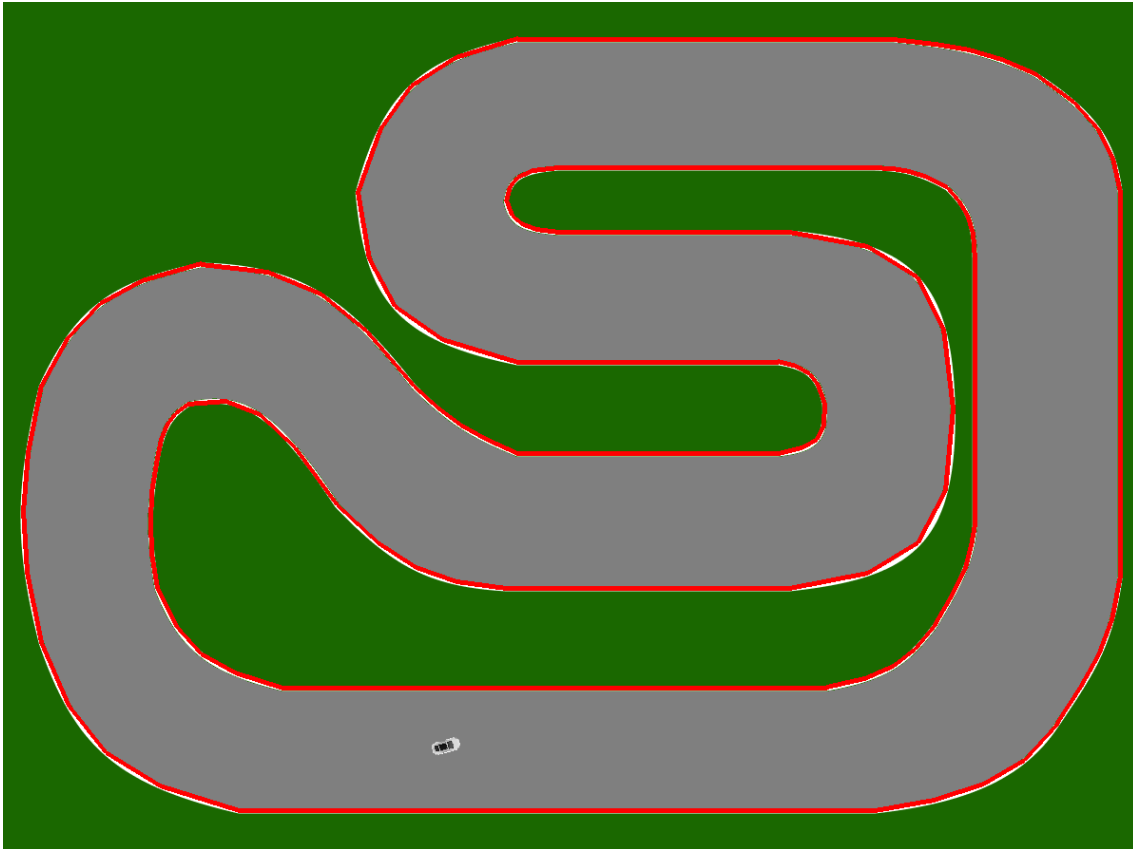


Abbildung 2.2: Visualisierung des Ergebnisses der prozeduralen Streckengenerierung. Die in Figma entworfenen, glatten Streckenbegrenzungen (weiß) werden durch eine Kette von kurzen, geraden Liniensegmenten (rot) approximiert, die die tatsächliche physikalische Wand für die Simulation bilden.

Für das Design der Rennstrecke und der grafischen Assets wurde der webbasierte Vektor-Grafikeditor Figma verwendet. Dieser Ansatz bot zwei entscheidende Vorteile: volle kreative Kontrolle über das Streckenlayout und die Möglichkeit, eine präzise und reproduzierbare physikalische Repräsentation der Umgebung zu generieren. Der Designprozess war zweistufig:

1. **Visuelles Design:** Zunächst wurde die gesamte Rennstrecke visuell in Figma gestaltet. Die inneren und äußeren Streckenbegrenzungen wurden dabei als komplexe, glatte Vektorpfade (in [Abbildung 2.2](#) als weiße Linien dargestellt) im SVG-Format gezeichnet.
2. **Prozedurale Konvertierung:** Um diese visuellen Kurven für die physikalische Kollisions- und Distanzmessung nutzbar zu machen, wurde ein eigenes Python-Skript

entwickelt. Dieses Skript liest die SVG-Daten ein und approximiert die glatten Kurven durch eine Sequenz von kurzen, geraden Liniensegmenten.

Das Ergebnis dieses Prozesses ist in [Abbildung 2.2](#) visualisiert: Die ursprünglich in Figma gezeichneten, glatten weißen Kurven werden durch eine Kette von verbundenen roten Liniensegmenten überlagert. Diese roten Segmente stellen die finale, für die Physik-Engine verwertbare Streckengeometrie dar. Diese prozedurale Umwandlung stellt sicher, dass die für die Simulation verwendete Geometrie sehr genau dem visuell gestalteten Layout entspricht. Die resultierenden Liniensegmente dienen als physikalische „Wände“ für die Kollisionserkennung und als Zielobjekte für die in [Abschnitt 6.2](#) beschriebene Ray-Casting-Implementierung der Distanzsensoren.

2.4.4 TensorFlow und Keras für Deep Learning

Für die Implementierung der neuronalen Netze, die das Herzstück der DQN- und DDQN-Agenten bilden, wurde das Open-Source-Framework TensorFlow von Google eingesetzt [34]. TensorFlow ist eine der führenden Plattformen für maschinelles Lernen und bietet die notwendigen Werkzeuge für die Definition, das Training und den Einsatz von tiefen neuronalen Netzen. Insbesondere wurde die hoch-levelige API `tf.keras` verwendet, die eine intuitive und schnelle Erstellung von Netzwerkarchitekturen ermöglicht. Die zentralen Vorteile von TensorFlow für diese Arbeit waren:

- **Automatischer Gradientenabstieg:** Die Fähigkeit, Gradienten automatisch zu berechnen (Automatic Differentiation), ist fundamental für die Optimierung der Netzwerkparameter während des Trainings.
- **Flexibilität und Skalierbarkeit:** Obwohl in dieser Arbeit hauptsächlich auf der CPU trainiert wurde, bietet TensorFlow die Möglichkeit, das Training nahtlos auf GPUs oder andere Beschleuniger zu verlagern.
- **Integration mit TensorBoard:** Die nahtlose Anbindung an das Visualisierungswerkzeug TensorBoard war für die Analyse des Trainingsverlaufs von entscheidender Bedeutung.

2.4.5 NumPy für numerische Operationen

Die Bibliothek NumPy (Numerical Python) bildete das numerische Fundament des gesamten Projekts. Sie stellt hocheffiziente, mehrdimensionale Array-Objekte bereit, die für schnelle mathematische Operationen unerlässlich sind. Im Rahmen dieser Arbeit wurde

NumPy intensiv für mehrere Kernaufgaben genutzt: Es diente zur Verwaltung des Replay Buffers, der als großes NumPy-Array implementiert wurde, und zur Repräsentation und Manipulation von Zustandsvektoren, Aktionen und Belohnungen. Darüber hinaus war NumPy entscheidend für die Durchführung der Vektor- und Matrixoperationen, die für die Berechnung der Zielwerte im Lernprozess der DQN- und DDQN-Agenten notwendig sind.

2.4.6 Visualisierung der Experimente mit TensorBoard und Plotly

Zur Überwachung und Analyse des Trainingsfortschritts wurden zwei verschiedene Werkzeuge zur Visualisierung eingesetzt, die jeweils unterschiedliche Zwecke erfüllten.

Für die Echtzeit-Überwachung während des Trainings wurde TensorBoard, das Visualisierungs-Toolkit von TensorFlow, verwendet. Es ermöglicht die grafische Darstellung von Metriken in Echtzeit und war unerlässlich, um den Verlauf von Trainingsgrößen wie der kumulierten Belohnung, dem Explorationsparameter Epsilon und insbesondere der durchschnittlichen maximalen Q-Wert-Schätzung live zu beobachten und das Verhalten der Agenten zu diagnostizieren.

Für die Erstellung der finalen Abbildungen und Graphen in dieser Arbeit kam die Python-Bibliothek Plotly zum Einsatz. Nach Abschluss der Experimente wurden die in CSV-Dateien gespeicherten Rohdaten mit Plotly verarbeitet, um hochwertige Visualisierungen zu erzeugen. Dieser zweistufige Prozess trennt die schnelle, explorative Analyse während der Entwicklung (TensorBoard) von der sorgfältigen, finalen Darstellung der Ergebnisse (Plotly).

2.4.7 Git und GitHub für Versionierung und Veröffentlichung

Um eine nachvollziehbare und wissenschaftlich saubere Arbeitsweise zu gewährleisten, wurde das gesamte Projekt mit dem Versionskontrollsystem Git verwaltet. Dies ermöglichte die lückenlose Nachverfolgung aller Änderungen am Quellcode und den Konfigurationsdateien. Um die Reproduzierbarkeit der in dieser Arbeit präsentierten Ergebnisse zu maximieren, wird der vollständige Quellcode auf der Plattform GitHub unter einer Open-Source-Lizenz öffentlich zur Verfügung gestellt¹.

¹Der Quellcode ist verfügbar unter: <https://github.com/Zickswag/bachelor-thesis>

Kapitel 3

Verwandte Arbeiten

Die wissenschaftliche Einordnung der vorliegenden Arbeit erfordert die Betrachtung von Studien aus drei zentralen Bereichen: erstens, die Anwendung von Reinforcement Learning in Simulationsumgebungen für autonome Fahrzeuge; zweitens, direkte empirische Vergleiche zwischen den Kernalgorithmen DQN und DDQN; und drittens, breitere Forschungsansätze zum Problem des Overestimation Bias.

3.1 Reinforcement Learning in Simulationsumgebungen

Die Entwicklung von RL-Agenten für Fahraufgaben wird häufig in spezialisierten Simulationsumgebungen durchgeführt. Hierbei sind zwei zentrale Designentscheidungen von besonderer Bedeutung, die im Folgenden diskutiert werden: die Wahl der Zustandsrepräsentation für den Agenten und der Realitätsgrad der Simulationsumgebung selbst.

3.1.1 End-to-End-Ansätze versus kompakte Zustandsrepräsentationen

Ein klassischer und weit verbreiteter Ansatz nutzt dabei rohe Bilddaten als Eingabe für den Agenten. Ein frühes Beispiel hierfür ist die Arbeit von Mnih et al. [3] selbst, die DQN auf Atari-Spielen mit reinen Pixel-Inputs zum Erfolg führten. Im Kontext des autonomen Fahrens wird dieser Ansatz in Studien wie der von Jaritz et al. [35] aufgegriffen, die DQN in der fotorealistischen Simulationsumgebung CARLA einsetzen. Im Gegensatz zu diesen Ansätzen, die eine hohe dimensionale Komplexität durch die Bildverarbeitung mit sich bringen, nutzt die vorliegende Arbeit eine bewusst gewählte, kompakte Zustandsrepräsentation. Diese basiert hauptsächlich auf 16 Distanzsensoren, was die Komplexität der zu lernenden Aufgabe signifikant reduziert und das Training auf die wesentlichen Fahrdynamiken fokussiert.

3.1.2 Realismus und Komplexität von Simulationsumgebungen

Eine wichtige Dimension bei der Betrachtung von Simulationsumgebungen ist der Grad des Realismus. Während einfache 2D-Umgebungen schnelle Iterationszyklen ermöglichen

chen, zielen fortgeschrittene Simulatoren wie CARLA oder TORCS (The Open Racing Car Simulator) darauf ab, eine hochgradig realitätsnahe Physik und Sensorik zu modellieren. In der Arbeit zur „TORCS Reinforcement Learning Competition“ von Loiacono et al. [36] wird eine jährliche Competition beschrieben, die als Benchmark für RL-basierte Fahragenten dient. Die Agenten müssen hier komplexe Fahrdynamiken, verschiedene Wetterbedingungen und den Verschleiß von Reifen meistern. Die Erwähnung dieser hochrealistischen Simulatoren dient der Einordnung der in dieser Arbeit verwendeten, bewusst einfacher gehaltenen Pygame-Umgebung. Der gewählte Ansatz legt den Fokus gezielt auf den grundlegenden Vergleich der Algorithmen, frei von den komplexen Störfaktoren einer vollumfänglichen Physiksimulation, und verdeutlicht den bewussten Trade-off zwischen Realismus und Kontrollierbarkeit. Diese Abwägung ist zentral, da eine zu starke Vereinfachung die Übertragbarkeit der Ergebnisse in die reale Welt erschweren kann, ein als *Sim-to-Real Gap* bekanntes Problem [37].

Methodisch deutlich näher an dieser Arbeit ist die Studie „Enhanced Deep Q-Learning for 2D Self-Driving Cars“ von Mustafina et al. [38]. Die Autoren entwickelten eine eigene 2D-Rennsimulation mittels Pygame und trainierten einen DQN-Agenten, dessen Entscheidungen auf den Daten von sieben Distanzsensoren basieren. Die vorliegende Arbeit greift diesen vielversprechenden Ansatz auf und erweitert ihn maßgeblich. Der hier verwendete Zustandsraum ist 18-dimensional und umfasst nicht nur eine höhere Anzahl an Distanzsensoren für eine präzisere Umfelderkennung, sondern schließt auch die aktuelle Geschwindigkeit und die Winkelgeschwindigkeit des Fahrzeugs mit ein. Dadurch erhält der Agent umfassendere Informationen über seine eigene Fahrdynamik, was zu fundierteren und stabileren Entscheidungen führen soll.

3.2 Empirische Vergleiche von DQN und DDQN

Das Fundament für die zentrale Hypothese dieser Arbeit liefert das Original-Paper „Deep Reinforcement Learning with Double Q-Learning“ von van Hasselt, Guez und Silver [5]. In dieser wegweisenden Veröffentlichung wurde der DDQN-Algorithmus eingeführt und dessen Überlegenheit gegenüber dem Standard-DQN auf den anspruchsvollen Atari-Benchmarks empirisch nachgewiesen. Die Autoren belegen, dass DDQN durch die Entkopplung von Aktionsauswahl und -bewertung die systematische Überschätzung der Q-Werte (Overestimation Bias) reduziert, was zu stabileren Trainingsprozessen und letztlich zu einer besseren Performanz führt.

Dass der Vergleich von DQN und DDQN ein nach wie vor aktuelles Forschungsthema ist, zeigt die Arbeit „A Study on the Impact of Obstacle Size on Training Models Based on DQN and DDQN“ [39]. In einer selbst entwickelten Umgebung mit Hindernissen bestä-

tigen die Autoren, dass DDQN insbesondere in Szenarien mit zunehmender Komplexität dem DQN-Algorithmus überlegen ist.

Die Frage, welcher Algorithmus überlegen ist, wird nicht nur im Kontext von Spielen, sondern auch in anderen Anwendungsfeldern untersucht. In „Mobile Robot Navigation Using Deep Reinforcement Learning“ [6] werden DQN und DDQN für die Aufgabe der autonomen Navigation eines mobilen Roboters in einer Umgebung mit statischen und dynamischen Hindernissen verglichen. Ihre Ergebnisse zeigen, dass DDQN zu einer signifikant geringeren Anzahl an Kollisionen und einem schnelleren Erreichen des Ziels führt. Die Untersuchung dieser Studie ist von besonderem Interesse, da sie die Relevanz der Forschungsfrage über die Spieledomäne hinaus unterstreicht. Sie zeigt, dass der Vergleich von DQN und DDQN auch in der Robotik fundamental ist und legt nahe, dass die in dieser Arbeit gewonnenen Erkenntnisse potenziell auf andere dynamische Steuerungsprobleme übertragbar sind.

3.3 Studien zur Mitigation des Overestimation Bias

Der DDQN-Algorithmus ist die bekannteste, aber bei weitem nicht die einzige vorgeschlagene Lösung zur Mitigation des Overestimation Bias und zur generellen Leistungssteigerung von DQN. Ein breites Spektrum an Verbesserungen wurde entwickelt, die oft orthogonal zueinander wirken und auf unterschiedliche Aspekte des Lernprozesses abzielen.

3.3.1 Dueling-Netzwerkarchitekturen als Alternative

Eine einflussreiche Architektur zur Verbesserung von DQN ist das Dueling Network, vorgestellt von Wang et al. in „Dueling Network Architectures for Deep Reinforcement Learning“ [40]. Dieser Ansatz modifiziert die Netzwerkarchitektur selbst. Anstatt einen einzelnen Q-Wert auszugeben, schätzt das Netzwerk zwei separate Ströme: den zustandsabhängigen Wert (State-Value, $V(s)$) und den vorteilsabhängigen Wert für jede Aktion (Advantage, $A(s, a)$). Diese werden am Ende zu den Q-Werten kombiniert. Die Kernidee ist, dass der Wert eines Zustands nicht zwingend von jeder Aktion abhängt. Die Betrachtung von Dueling Networks ist insofern relevant, als dass sie einen orthogonalen Lösungsansatz zum Overestimation Bias darstellt. Während DDQN die Update-Regel modifiziert, passt die Dueling-Architektur die Netzwerkstruktur selbst an.

3.3.2 Orthogonale Ansätze zur Lernstabilität und Effizienz

Ein zentraler Aspekt zur Steigerung der Stabilität ist die Qualität der Zielwerte (Target-Values), die für das Update des Q-Netzwerks verwendet werden. Hier schlagen Anschel

et al. mit *Averaged-DQN* [41] eine Methode vor, die nicht nur das letzte Target-Netzwerk nutzt, sondern die Gewichte mehrerer vergangener Target-Netzwerke mittelt. Diese Mittelung reduziert die Varianz in den Ziel-Schätzungen und kann so zu einem ruhigeren und stabileren Lernprozess führen.

Eine weitere fundamentale Verbesserung des DQN-Frameworks ist das *Prioritized Experience Replay*, vorgestellt von Schaul et al. [42]. Anstatt Übergänge uniform aus dem Replay-Buffer zu ziehen, gewichtet dieser Ansatz sie anhand ihres Temporal-Difference (TD) Fehlers. Übergänge mit hohem Fehler, bei denen der Agent also eine große Überraschung erlebt, werden häufiger für das Training ausgewählt. Dadurch lernt der Agent effizienter von wichtigen oder seltenen Ereignissen und die Daten-Effizienz des Trainings wird signifikant gesteigert.

Einen grundlegend anderen Weg zur Steigerung der Lernstabilität schlägt das *Distributional Reinforcement Learning* vor. In ihrer wegweisenden Arbeit zu C51 argumentieren Bellemare et al. [43], dass die Modellierung des reinen Erwartungswerts der Q-Funktion zu Informationsverlust führt. Stattdessen schlagen sie vor, die gesamte Verteilung der zukünftigen Belohnungen als Histogramm zu schätzen. Dieser Ansatz liefert ein reichhaltigeres Lernsignal, reduziert Schätzfehler und führt nachweislich zu einer robusteren Performanz. Auch die Exploration, ein kritischer Aspekt des RL-Prozesses, wurde optimiert. Anstatt der einfachen ϵ -greedy-Strategie führen Fortunato et al. mit *Noisy Networks* [44] eine Methode ein, die erlerntes, zustandsabhängiges Rauschen in die Gewichte des Netzwerks injiziert. Dies ermöglicht eine deutlich effizientere und gezieltere Exploration des Zustands-Aktions-Raums im Vergleich zu zufälligen Aktionen.

3.3.3 Kombinatorische Ansätze und Abgrenzung

Ein Meilenstein in der Weiterentwicklung von DQN ist der „Rainbow“-Algorithmus von Hessel et al. [45]. Dieses Paper kombiniert sechs der genannten Verbesserungen – darunter DDQN, Dueling Networks und Prioritized Replay – zu einem einzigen, extrem leistungsfähigen Agenten. Während sich die vorliegende Arbeit bewusst auf den fundamentalen Vergleich von DQN und DDQN konzentriert, um deren isolierten Effekt in der Rennspielumgebung zu analysieren, demonstriert Rainbow eindrucklich das Potenzial, das in der Kombination verschiedener Techniken liegt. Um den Effekt der Double Q-Learning Update-Regel isoliert zu untersuchen, werden in dieser Arbeit bewusst keine weiteren Verbesserungen wie Prioritized Replay, Dueling Architectures, Distributional RL oder Noisy Networks implementiert. Die Forschung hat sich seit Rainbow weiterentwickelt zu noch allgemeineren Agenten wie Agent57 [46], die zusätzlich Mechanismen für episodisches Gedächtnis und intrinsische Motivation integrieren. Eine solche Komplexität liegt jedoch bewusst außerhalb des Rahmens dieser Arbeit, die auf eine klare und isolierte Analyse ab-

zielt. Die hier nicht implementierten Techniken stellen jedoch vielversprechende Ansätze für zukünftige Erweiterungen dar.

3.3.4 Abgrenzung und methodischer Beitrag der Arbeit

Die vorliegende Arbeit grenzt sich in mehreren wesentlichen Punkten von bestehenden Untersuchungen im Bereich des Reinforcement Learning für Fahraufgaben ab.

Erstens wird bewusst ein reduktionistischer Ansatz verfolgt. Im Gegensatz zu Studien, die auf hochrealistische 3D-Simulatoren wie *CARLA* oder *TORCS* setzen und dabei meist rohe Bilddaten verarbeiten [35, 36], wird hier eine schlanke 2D-Simulation in Pygame genutzt. Der Agent erhält einen wohldefinierten, niedrigdimensionalen Zustandsvektor anstelle von Pixel-Eingaben. Diese methodische Wahl eliminiert die komplexe Aufgabe des Repräsentationslernens aus visuellen Daten und ermöglicht eine fokussierte Analyse des *reinen algorithmischen Unterschieds* zwischen DQN und DDQN. Damit wird eine klare Attribution der Leistungsgewinne zur Update-Regel des Algorithmus ermöglicht, eine Trennung, die in End-to-End-Ansätzen oft schwierig ist. Die Arbeit erweitert dabei methodisch verwandte 2D-Ansätze wie den von Mustafina et al. [38] durch einen reichhaltigeren Zustandsraum, der dem Agenten mehr Informationen über die eigene Fahrdynamik zur Verfügung stellt.

Zweitens legt die Arbeit ein besonderes Augenmerk auf statistische Belastbarkeit und methodische Transparenz. In Anlehnung an die Empfehlungen von Henderson et al. [47] zur Adressierung der hohen Varianz in RL-Experimenten werden die Ergebnisse nicht auf Basis einzelner Läufe, sondern durch die Aggregation von fünf unabhängigen Trainingsläufen mit unterschiedlichen Seeds präsentiert. Alle zentralen Metriken werden konsistent mit 95 %-Konfidenzintervallen visualisiert, um die Stabilität und Signifikanz der Ergebnisse bewertbar zu machen. Weiterhin wird die Wahl der Hyperparameter, insbesondere der Lernrate und der Trainingsdauer, explizit durch ein Vor-Experiment und eine Auseinandersetzung mit der Literatur begründet [3, 48], anstatt etablierte Werte unreflektiert zu übernehmen.

Drittens erfolgt eine direkte Quantifizierung des Overestimation Bias. Während viele Studien, einschließlich des fundamentalen Papers von van Hasselt et al. [5], den reduzierten Bias primär indirekt über die verbesserte Performanz des Agenten inferieren, wird in dieser Arbeit die Entwicklung der Q-Wert-Schätzungen direkt verfolgt und visualisiert. Durch die regelmäßige Messung des durchschnittlichen maximalen Q-Wertes auf einer festen Stichprobe von Zuständen wird der systematische Unterschied zwischen den Schätzungen von DQN und DDQN explizit nachgewiesen und mit der beobachteten Lernperformanz korreliert.

Schließlich wird der gesamte Code- und Datensatzstand inklusive der Konfigurationen für die Experimente unter einer Open-Source-Lizenz reproduzierbar veröffentlicht. Damit adressiert die Arbeit die in der RL-Forschung häufig kritisierte Replizierbarkeitslücke und schafft eine belastbare Grundlage für künftige Forschung, etwa für den Einsatz weiterführender Verfahren wie Prioritized Replay oder Dueling Architectures [45] auf dem hier vorgestellten Setting.

Kapitel 4

Problemstellung und Anforderungen

Dieses Kapitel legt die wissenschaftliche Problemstellung dar, die dieser Arbeit zugrunde liegt, und leitet daraus die konkreten Anforderungen an das experimentelle System ab. Es bildet die logische Brücke zwischen den in Kapitel 2 vorgestellten theoretischen Grundlagen und dem in Kapitel 5 beschriebenen methodischen Konzept.

4.1 Problemstellung

Die Kombination von Reinforcement Learning mit tiefen neuronalen Netzen ermöglicht es, komplexe Steuerungsaufgaben zu lösen. Die dabei verwendeten Wertefunktionen sind jedoch keine exakten Wahrheiten, sondern Schätzungen, die durch stochastische Gradientenverfahren gelernt werden. Diese Schätzungen sind fehlerbehaftet, was zu systematischen Problemen im Lernprozess führen kann.

4.1.1 Das Kernproblem: Der Overestimation Bias in DQN

Wie in Kapitel 2.2.3 bereits erläutert, leidet der Standard-DQN-Algorithmus unter einem signifikanten, systematischen Problem: dem Overestimation Bias [4]. Dieses Problem entsteht durch den Maximierungsoperator ($\max$) in der Update-Regel für den Zielwert (siehe Gleichung 2.8). Da die Q-Werte verrauschte Schätzungen sind, bevorzugt der $\max$ -Operator tendenziell Aktionen, deren Werte zufällig nach oben vom wahren Wert abweichen. Diese positive Verzerrung propagiert sich durch die Bellman-Updates im System und führt zu einer kumulativen und oft erheblichen Überschätzung der wahren Aktionswerte [5].

Im Kontext des hier entwickelten Rennspiels könnte dies bedeuten, dass der DQN-Agent eine riskante Aktion, wie das sehr späte Bremsen vor einer Kurve, systematisch zu optimistisch bewertet. Ein zufällig hoher Q-Wert-Schätzer könnte den Agenten dazu verleiten, diese gefährliche Strategie zu bevorzugen, obwohl sie im Durchschnitt zu häufigeren Kollisionen und damit zu einer schlechteren Gesamtperformanz führt.

4.1.2 Der postulierte Lösungsansatz: Double Q-Learning

Als theoretische Lösung für dieses Problem wurde der Double-Q-Learning-Ansatz vorgeschlagen und für tiefe neuronale Netze im DDQN-Algorithmus adaptiert [5]. Die Kernidee von DDQN ist die Entkopplung von Aktionsauswahl und Aktionsbewertung. Wie in Gleichung 2.9 gezeigt, wird das Online-Netzwerk zur Auswahl der besten Aktion im Folgezustand verwendet, während das Target-Netzwerk deren Wert liefert. Dies reduziert die Wahrscheinlichkeit, dass derselbe positive Schätzfehler eine Aktion sowohl zur Auswahl als auch zur Bewertung empfiehlt, was den Overestimation Bias mitigieren soll.

4.1.3 Forschungslücke und Hypothesen

Während die Überlegenheit von DDQN gegenüber DQN auf den hochdimensionalen, pixel-basierten Atari-Benchmarks empirisch nachgewiesen wurde [5], bleibt die Frage offen, wie sich dieser Vorteil unter fundamental anderen Bedingungen manifestiert. Die meisten Vergleiche konzentrieren sich auf das End-to-End-Lernen aus Bildern, bei dem das Rauschen in den Q-Wert-Schätzungen sowohl aus dem Lernprozess selbst als auch aus der komplexen Aufgabe der Merkmalsextraktion resultiert. Die zentrale Forschungslücke dieser Arbeit liegt daher in der Untersuchung des Overestimation Bias unter „Laborbedingungen“ mit einem niedrigdimensionalen, merkmalsbasierten Zustandsraum. In einem solchen Setup wird die Repräsentationslern-Aufgabe vom Problem entkoppelt, wodurch der Effekt der reinen Werteschätzungs-Dynamik isoliert untersucht werden kann. Zudem wird dies im spezifischen Kontext einer Rennspielumgebung analysiert, deren kontinuierliche Fahrdynamik und verzögerte Belohnungen eine Herausforderung für eine stabile Werteschätzung darstellen. Diese Fokussierung auf die reine Werteschätzungs-Dynamik ermöglicht es zudem, eine minimalistische Netzwerkarchitektur zu verwenden, um den Einfluss der Modellkapazität zu minimieren und die algorithmischen Unterschiede klarer herauszuarbeiten. Dies führt zur folgenden, präzisierten Forschungsfrage: *Führt die durch DDQN implementierte Kompensation des Overestimation Bias auch in einer merkmalsbasierten Rennspielumgebung zu einer messbar überlegenen und stabileren Lern-Performanz im Vergleich zu Standard-DQN?*

Aus dieser Frage werden zwei zentrale, überprüfbare Hypothesen abgeleitet:

- H1:** Der DDQN-Agent wird im Vergleich zum DQN-Agenten systematisch niedrigere durchschnittliche maximale Q-Werte aufweisen.
- H2:** Aufgrund der genaueren Werteschätzung wird der DDQN-Agent eine überlegene Policy erlernen, die sich in einer höheren kumulativen Belohnung pro Episode manifestiert.

4.2 Anforderungen

Um die aufgestellten Hypothesen empirisch zu überprüfen, müssen ein Softwaresystem und ein experimenteller Aufbau entwickelt werden, die die folgenden funktionalen und nicht-funktionalen Anforderungen erfüllen.

4.2.1 Funktionale Anforderungen

Tabelle 4.1: Anforderung F1 – Implementierung der Rennspielumgebung

Nr	F1	Art	F	Prio	Hoch
Titel		Implementierung der Rennspielumgebung			
Herkunft		Eigene Anforderung aus der Problemstellung			
Beschreibung		Eine Rennspielumgebung ist implementiert			
Fit-Kriterium		Die in Kapitel 5.1 beschriebene Zustands- und Aktionsräume sowie Belohnungsstrukturen sind exakt umgesetzt			

Tabelle 4.2: Anforderung F2 – Implementierung des DQN-Agenten

Nr	F2	Art	F	Prio	Hoch
Titel		Implementierung des DQN-Agenten			
Herkunft		Standard-Algorithmus nach Mnih et al. (2015)			
Beschreibung		Ein DQN-Agent ist implementiert			
Fit-Kriterium		Agent folgt strikt der Update-Logik von Standard-DQN, wie in Gleichung 2.8 definiert			

Tabelle 4.3: Anforderung F3 – Implementierung des DDQN-Agenten

Nr	F3	Art	F	Prio	Hoch
Titel	Implementierung des DDQN-Agenten				
Herkunft	Standard-Algorithmus nach van Hasselt et al. (2016)				
Beschreibung	Ein DDQN-Agent ist implementiert				
Fit-Kriterium	Agent folgt strikt der Update-Logik von DDQN, wie in Gleichung 2.10 definiert				

Tabelle 4.4: Anforderung F4 – Durchführung und Protokollierung von Trainingsläufen

Nr	F4	Art	F	Prio	Hoch
Titel	Durchführung und Protokollierung von Trainingsläufen				
Herkunft	Eigene Anforderung für die Evaluation				
Beschreibung	Das System ist in der Lage, die Agenten zu trainieren und dabei Metriken zu protokollieren				
Fit-Kriterium	Nach Abschluss eines Trainingslaufs existiert eine Log-Datei im CSV-Format				

4.2.2 Nicht-funktionale Anforderungen

Tabelle 4.5: QAS NF1 – Vergleichbarkeit der Algorithmen

Nr	NF1	Art	QAS	Prio	Hoch
Titel	Vergleichbarkeit der Algorithmen				
Quelle	Der Forschende / Gutachter				
Stimulus	Startet einen DQN-, sowie einen DDQN-Trainingslauf				
Artefakt	Das Experiment-System				
Umgebung					
Das System befindet sich im initialen Zustand vor dem Training					
Antwort					
Zwei separate Konfigurationsdateien und gespeicherte Code-Struktur					
Maß für Antwort					
Die Konfigurationsdateien sind bis auf den Agenttyp identisch und die Agenten werden in der exakt gleichen Umgebung eingesetzt					

Tabelle 4.6: QAS NF2 – Reproduzierbarkeit der Ergebnisse

Nr	NF2	Art	QAS	Prio	Hoch
Titel	Reproduzierbarkeit der Ergebnisse				
Quelle	Ein externer Gutachter / Dritter Forscher				
Stimulus	Startet zwei Trainingsläufe desselben Algorithmus mit identischen Hyperparametern, Seed sowie Betriebssystem				
Artefakt	Das Experiment-System				
Umgebung					
Das System befindet sich im initialen Zustand vor dem Training					
Antwort					
Zwei separate CSV-Dateien					
Maß für Antwort					
Die Dateien sind identisch					

Tabelle 4.7: QAS NF3 – Vollständigkeit der Evaluationsmetriken

Nr	NF3	Art	QAS	Prio	Hoch
Titel	Vollständigkeit der Evaluationsmetriken				
Quelle	Der Forschende / Gutachter				
Stimulus	Startet einen Trainingslauf				
Artefakt	Das Experiment-System				
Umgebung					
Das System befindet sich im initialen Zustand vor dem Training					
Antwort					
Eine CSV-Datei					
Maß für Antwort					
Vorhandensein der Metriken für den Score, die gesamten Trainingsschritte und Q-Werte über alle Episoden					

Tabelle 4.8: QAS NF4 – Angemessener Zeitrahmen für das Training

Nr	NF4	Art	QAS	Prio	Mittel
Titel		Angemessener Zeitrahmen für das Training			
Quelle		Der Forschende / Gutachter			
Stimulus		Startet einen vollständigen Trainingslauf über 50 Millionen Schritte			
Artefakt		Das Experiment-System			
Umgebung					
Das System ist auf der definierten Entwicklungs-Hardware (Apple M4 Pro) einsatzbereit					
Antwort					
Der Trainingslauf ist erfolgreich abgeschlossen					
Maß für Antwort					
Weniger als 48 Stunden					

Kapitel 5

Konzept und Methodik

Um die in Kapitel 1.2 formulierte Forschungsfrage zu beantworten und die zentrale Hypothese zu überprüfen, dass Double Deep Q-Networks (DDQN) den Overestimation Bias von Deep Q-Networks (DQN) in der entwickelten Rennspielumgebung reduzieren und dadurch zu einer überlegenen Performanz führen, wird ein streng kontrolliertes, empirisches Experiment durchgeführt. Dieses Kapitel beschreibt detailliert das methodische Vorgehen, beginnend mit der Formalisierung der Umgebung als Markov-Entscheidungsprozess, über die detaillierte Beschreibung der Trainingsinfrastruktur und des experimentellen Designs bis hin zu den spezifischen Metriken, die zur Evaluation der Algorithmen herangezogen werden.

5.1 Formalisierung der Rennspielumgebung

Die entwickelte Rennspielumgebung wird im Rahmen des Reinforcement Learning als Markov-Entscheidungsprozess (MDP) modelliert, wie in Abschnitt 2.1.2 beschrieben. Die Schlüsselkomponenten sind präzise definiert, um eine reproduzierbare und nachvollziehbare Grundlage für das Agententraining zu schaffen.

5.1.1 Zustandsraum ($\mathcal{S}$)

Der Zustand $s_t \in \mathcal{S}$ zum Zeitpunkt t liefert dem Agenten eine Repräsentation seiner unmittelbaren Umgebung und seines eigenen dynamischen Zustands. Im Gegensatz zu Ansätzen, die auf hochdimensionalen Pixeldaten basieren [3], wird hier eine kompakte, merkmalsbasierte Zustandsrepräsentation verwendet. Dies reduziert die Komplexität des Lernproblems und fokussiert das Training auf die wesentlichen Fahrdynamiken. Der Zustand s_t ist ein Vektor mit 18 kontinuierlichen Werten:

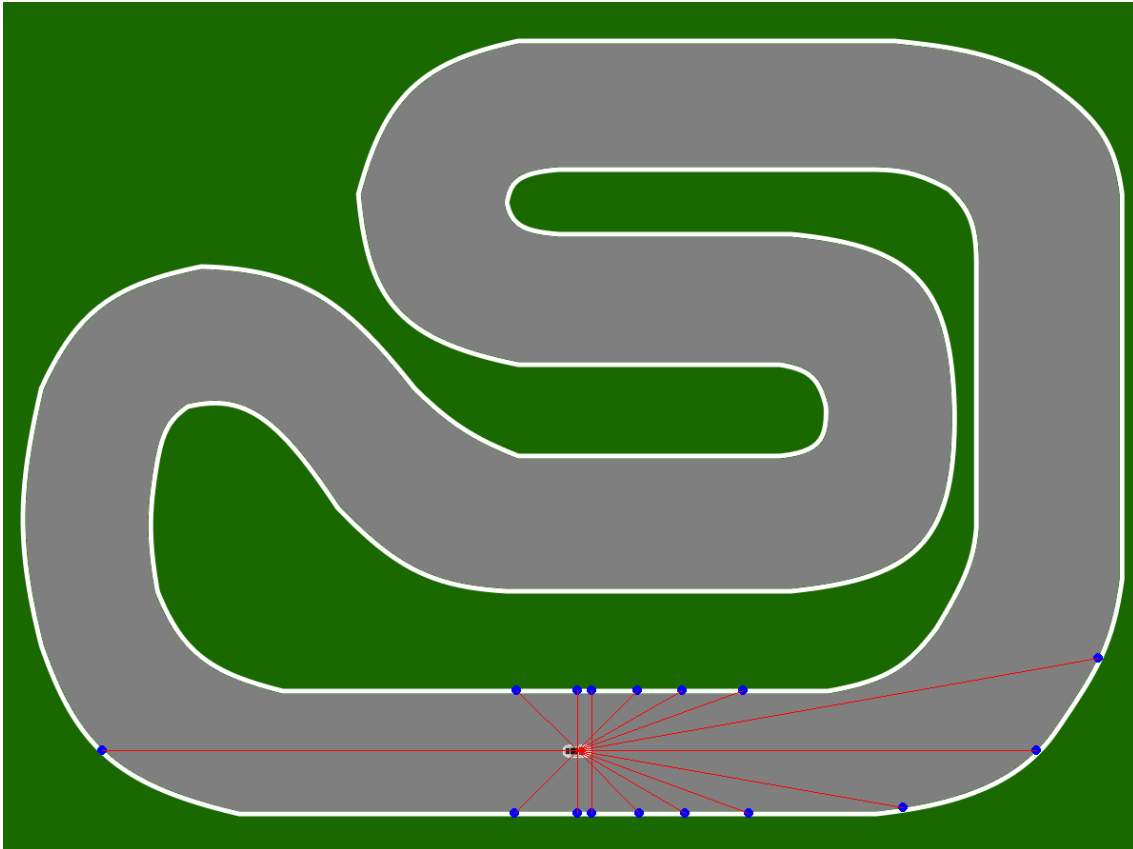


Abbildung 5.1: Die Wahrnehmung des Agenten durch 16 Distanzsensoren. Die roten Linien repräsentieren die vom Fahrzeug ausgehenden Sensorstrahlen (Ray-Casts). Die blauen Punkte markieren die nächstgelegenen Schnittpunkte mit der physikalischen Streckenwand. Die Längen dieser Strahlen bilden den primären Input für das neuronale Netzwerk.

- **16 Distanzsensoren:** Sechzehn Strahlen, die vom Fahrzeug ausgehen, messen die Distanz zur nächsten Streckenbegrenzung. Dieses Vorgehen, visualisiert in [Abbildung 5.1](#), emuliert die Funktionsweise von LIDAR-Systemen [49]. Die roten Linien stellen die Sensorstrahlen dar, die in festen Winkeln vom Agenten ausgehen, während die blauen Punkte die berechneten Schnittpunkte mit der Streckenwand markieren. Die Längen dieser Strahlen bilden die ersten 16 Werte des Zustandsvektors und geben dem Agenten eine 360-Grad-Wahrnehmung seiner Position. Eine spezifische, nicht-uniforme Verteilung der Strahlen, mit einer höheren Dichte im vorderen Bereich und an den Fahrzeugecken, wurde gewählt, um kritische Bereiche für die Kollisionsvermeidung präziser abzutasten.
- **Lineare Geschwindigkeit:** Die aktuelle skalare Geschwindigkeit des Fahrzeugs. Dieses Merkmal ist entscheidend für das Timing von Brems- und Beschleunigungsmanövern.

- **Winkelgeschwindigkeit:** Die Änderungsrate des Bewegungsvektors des Fahrzeugs. In der implementierten Physik folgt der Bewegungsvektor der Fahrzeugausrichtung nur mit einer gewissen Trägheit, was ein Drift-Verhalten simuliert. Die Winkelgeschwindigkeit quantifiziert die Stärke dieses Drifts und ist für den Agenten essenziell, um Kurven vorausschauend zu fahren und ein Übersteuern zu vermeiden. Die Einbeziehung solcher ego-dynamischer Merkmale ist eine gängige Praxis in der Robotik und bei autonomen Fahraufgaben [36, 50], um dem Agenten ein besseres internes Modell seiner eigenen Bewegung zu ermöglichen.

Erfüllung der Markov-Eigenschaft

Eine grundlegende Annahme vieler RL-Algorithmen ist die Markov-Eigenschaft, die besagt, dass der aktuelle Zustand s_t alle Informationen enthält, die zur Vorhersage des Folgezustands s_{t+1} nach einer Aktion a_t notwendig sind. In der Praxis wird diese Eigenschaft in komplexen Systemen oft nur angenähert. Dies kann auf nicht erfasste Physik (z.B. Reifentemperatur), stochastische externe Einflüsse oder die Diskretisierung von Zeit und Raum zurückzuführen sein [7].

In der für diese Arbeit entwickelten Simulationsumgebung ist die Umgebung deterministisch, es gibt keine externen Störfaktoren, und die gewählten Zustandsmerkmale (Positionen, Orientierung, lineare und Winkelgeschwindigkeit) beschreiben die Systemdynamik vollständig. Daher wird davon ausgegangen, dass der Zustandsvektor eine ausreichende Statistik darstellt, die alle relevanten Informationen der Vergangenheit für die zukünftige Entscheidungsfindung komprimiert, was der Definition der Markov-Eigenschaft entspricht [7].

Normalisierung des Zustandsraums

Vor der Übergabe an das neuronale Netzwerk werden alle 18 Merkmale des Zustandsvektors normalisiert. Dies ist ein entscheidender Schritt, da neuronale Netze am effizientesten lernen, wenn die Eingabewerte in einem ähnlichen, kleinen Wertebereich liegen, typischerweise $[0, 1]$ oder $[-1, 1]$ [51]. Die Normalisierung verhindert, dass Merkmale mit von Natur aus großen Wertebereichen (wie die Sensordistanzen) den Lernprozess übermäßig dominieren. Konkret werden hierfür die 16 Distanzmessungen auf den Bereich $[0, 1]$ skaliert, wobei 0 eine maximale Distanz und 1 eine unmittelbare Nähe zur Wand repräsentiert. Die lineare Geschwindigkeit wird, basierend auf der definierten maximalen Vor- und Rückwärtsgeschwindigkeit, auf den Bereich $[-1, 1]$ normalisiert. Ebenso wird die Winkelgeschwindigkeit auf $[-1, 1]$ skaliert, was auf der physikalisch möglichen maximalen Rotationsrate pro Zeitschritt basiert.

5.1.2 Aktionsraum ($\mathcal{A}$)

Der Aktionsraum $\mathcal{A}$ ist diskret und besteht aus fünf möglichen Aktionen, die der Agent in jedem Zeitschritt t ausführen kann:

1. **Nichts tun**
2. **Beschleunigen**
3. **Bremsen**
4. **Nach links lenken**
5. **Nach rechts lenken**

Diese grundlegenden Aktionen ermöglichen eine vollständige Kontrolle über das Fahrzeug innerhalb der Spielphysik. Der Agent lernt eine Policy $\pi(a_t|s_t)$ zu entwickeln, die für jeden Zustand s_t die optimale Aktion a_t auswählt. Der Aktionsraum $\mathcal{A}$ ist bewusst diskret gewählt, um die direkte Anwendung von wertbasierten Deep-RL-Algorithmen wie DQN und dessen Erweiterungen zu ermöglichen. Während kontinuierliche Aktionsräume eine feinere Steuerung erlauben würden, erfordern sie policy-basierte oder Actor-Critic-Methoden (z.B. Deep Deterministic Policy Gradient (DDPG), Soft Actor-Critic (SAC)) [52]. Die Diskretisierung vereinfacht das Lernproblem und erlaubt einen fokussierten Vergleich der Mechanismen zur Q-Wert-Schätzung, was im Zentrum dieser Arbeit steht.

5.1.3 Belohnungsfunktion (R)

Der Agent erhält ausschließlich bei zwei Schlüsselereignissen eine Belohnung: Für das erfolgreiche Durchfahren eines Checkpoints wird eine positive Belohnung von +1 vergeben, während eine Kollision mit der Streckenbegrenzung zu einer negativen Belohnung von -1 führt. In allen anderen Zeitschritten erfolgt keine Belohnung.

Wie in [Abbildung 5.2](#) dargestellt, ist die Strecke mit einer Reihe von Checkpoints (schwarze Linien) ausgestattet, die den Fortschritt des Agenten messen. Der als nächstes zu erreichende Checkpoint wird dabei dynamisch ermittelt und zur Orientierung rot hervorgehoben. Diese +1/-1-Struktur wurde einer strafbetonen Struktur (z.B. mit hohen negativen Werten) vorgezogen, um die Ziel-Q-Werte in einem kleinen, stabilen Bereich zu halten. Große, ungebundene Belohnungen können zu instabilen und stark oszillierenden Zielwerten führen, was das Training mit Funktionsapproximatoren erschwert und Techniken wie Reward Clipping oder Gradient Clipping erfordern würde, um die Konvergenz zu sichern [3]. Der gewählte Ansatz zielt darauf ab, die Stabilität des Trainings ohne solche zusätzlichen Mechanismen zu maximieren.

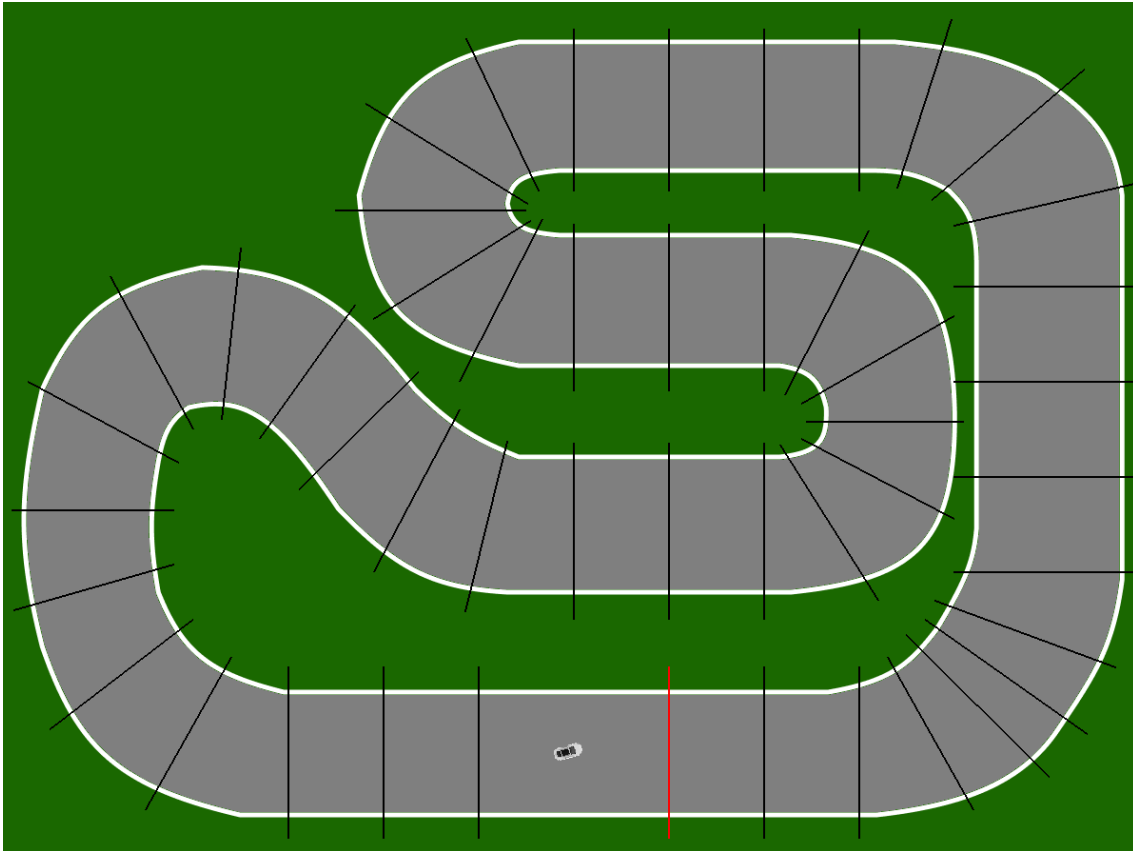


Abbildung 5.2: Das Checkpoint-System der Simulationsumgebung. Die schwarzen Linien markieren alle verfügbaren Checkpoints. Die rote Linie visualisiert den nächsten zu erreichenden Checkpoint für den Agenten. Das Erreichen dieses Ziels wird mit einer positiven Belohnung versehen.

5.1.4 Episoden-Terminierung

Eine Episode endet unter einer von drei alternativen Bedingungen. Erstens, im Falle einer *Kollision* des Fahrzeugs mit einer Streckenbegrenzung. Zweitens, durch einen *Fortschritts-Timeout*, wenn der Agent für 100 aufeinanderfolgende Schritte keinen neuen Checkpoint erreicht. Drittens, durch einen *Längen-Timeout*, falls die Episode eine maximale Länge von 1.000 Schritten überschreitet.

Die Timeout-Bedingungen sind entscheidend, um zu verhindern, dass der Agent in sub-optimalen Zyklen (z.B. im Kreis fahren) stecken bleibt und das Training blockiert. Die spezifischen Werte von 100 Schritten für den Fortschritts-Timeout und 1.000 Schritten für die maximale Episodenlänge wurden so gewählt, dass sie dem Agenten ausreichend Zeit für die Erkundung zwischen zwei Checkpoints lassen, aber stagnierendes Verhalten zuverlässig unterbinden.

5.2 Experimentelles Design

Um einen fairen und wissenschaftlich fundierten Vergleich zwischen DQN und DDQN sicherzustellen, wird ein kontrolliertes Experiment durchgeführt. Das Design orientiert sich eng an der Methodik von van Hasselt et al. [5], um die Vergleichbarkeit mit etablierten Forschungsergebnissen zu gewährleisten.

5.2.1 Netzwerkarchitektur und Hyperparameter

Der Kern des Experiments besteht darin, alle externen Faktoren und Hyperparameter für beide Algorithmen identisch zu halten. Der einzige Unterschied zwischen den experimentellen Gruppen ist die mathematische Formulierung der Update-Regel für den Zielwert, wie in Gleichung 2.8 für DQN und Gleichung 2.9 für DDQN beschrieben.

Netzwerkarchitektur

Sowohl der DQN- als auch der DDQN-Agent verwenden exakt dieselbe neuronale Netzwerkarchitektur, um einen fairen Vergleich zu gewährleisten. Zum Einsatz kommt ein vollständig verbundenes Netzwerk (Multi-Layer Perceptron (MLP)) mit folgender Struktur:

- **Eingabeschicht:** 18 Neuronen, entsprechend der Dimension des normalisierten Zustandsvektors.
- **Versteckte Schichten:** Drei versteckte Schichten mit jeweils 18 Neuronen.
- **Ausgabeschicht:** 5 Neuronen mit linearer Aktivierung, die jeweils den geschätzten Q-Wert für eine der fünf möglichen Aktionen repräsentieren.
- **Aktivierungsfunktion:** Als Aktivierungsfunktion für alle versteckten Schichten wird die Rectified Linear Unit (ReLU) verwendet.

Die Wahl der Netzwerkarchitektur ist eine fundamentale Designentscheidung, die den Bias-Variance-Tradeoff direkt beeinflusst [51]. Ein Netzwerk mit zu geringer Kapazität (zu wenige Neuronen oder Schichten) kann die Komplexität der optimalen Q-Funktion möglicherweise nicht abbilden (hoher Bias), während ein Netzwerk mit zu hoher Kapazität Gefahr läuft, Rauschen in den Trainingsdaten zu über-fitten und schlecht zu generalisieren (hohe Varianz). Die gewählte Architektur ist das Ergebnis einer bewussten Abwägung dieser Faktoren. Die Entscheidung für eine tiefe Architektur mit drei versteckten Schichten basiert auf der Notwendigkeit, hierarchische Merkmale zu extrahieren. Es wird angenommen, dass ein tiefes Netzwerk in der Lage ist, eine Abstraktionshierarchie zu erlernen: Die erste Schicht könnte grundlegende Muster aus den Sensor- und Geschwindigkeitsdaten

lernen (z.B. „Nähe zur Wand links“), die nachfolgenden Schichten diese zu kontextuellen Situationen kombinieren (z.B. „scharfe Linkskurve bei hoher Geschwindigkeit“) und die letzte Schicht diese Situationsbewertung in eine finale Aktionsbewertung umsetzt. Diese Fähigkeit zur hierarchischen Merkmalsextraktion ist für komplexe Steuerungsaufgaben unerlässlich und ein Kernprinzip des Deep Learning [51].

Im Gegensatz zu pixelbasierten Agenten, die große Netze zur Merkmalsextraktion benötigen (z.B. mit 256 oder 512 Neuronen), operiert der Agent hier auf einem niedrig-dimensionalen, merkmalsbasierten Zustandsvektor. Für solche Probleme sind schlankere Architekturen (mit bspw. 64 Neuronen [38]) die etablierte Norm. Die Wahl der Breite des Netzwerks dieser Arbeit mit nur 18 Neuronen pro Schicht folgt dem Prinzip der Sparsamkeit und dient als starke Regularisierung, um eine Überanpassung an den Replay Buffer zu vermeiden und das Erlernen einer generalisierbaren Strategie zu fördern [51].

Hyperparameter und Trainingsprozedur

Um eine gute Grundlagenforschung zu gewährleisten, basiert das für diese Arbeit gewählte Set an Hyperparametern fundamental auf dem von Mnih et al. [3] etablierten Rahmen. Dieses Referenz-Setup wurde jedoch an mehreren Schlüsselstellen systematisch an moderne Konventionen sowie die spezifische Problemstellung dieser Arbeit angepasst, um einen fairen und robusten Vergleich zu ermöglichen. Die resultierende Konfiguration, die für alle nachfolgenden Experimente verwendet wird, ist detailliert in Tabelle 5.1 dokumentiert. Eine wichtige methodische Anmerkung betrifft den Vergleich der Zeitschritte. Im Gegensatz zum Referenz-Setup, das eine Aktionswiederholung von 4 („Frame Skipping“) verwendet, interagiert der Agent in dieser Arbeit bei jedem einzelnen Zeitschritt mit der Umgebung. Parameter, die im Original-Paper in „Frames“ angegeben sind (z.B. die Größe des Replay Buffers), wurden daher auf die entsprechende Anzahl von Agenten-Schritten umgerechnet, um eine faire konzeptionelle Grundlage zu schaffen.

Ein Großteil der Parameter wurde direkt aus dem wegweisenden DQN-Referenz-Setup [3] übernommen. Dazu gehören der Diskontfaktor (γ), die effektive Größe des Replay Buffers (250.000 einzelne Interaktionsschritte (Transitionen)), die Frequenz der Target-Netzwerk-Updates (10.000 Schritte) und die Parameter der ϵ -greedy-Strategie.

Bei der Festlegung der Gesamttrainingsdauer wurde eine bemerkenswerte Inkonsistenz in der wegweisenden Literatur identifiziert. Die Originalarbeit zu DQN gibt eine Trainingsdauer von „50 million frames“ an [3, S. 6]. In der direkten Nachfolgearbeit zu Double DQN, deren explizites Ziel ein fairer Vergleich ist, geben die Autoren an, ihre Experimente unter den „exact conditions described by Mnih et al. (2015)“ [5, S. 4] durchgeführt zu haben. Im Text ist hier auf der gleichen Seite jedoch von „200M frames“ die Rede. Diese Diskrepanz wird durch die unklare Verwendung der Terminologie weiter erschwert. Wäh-

Tabelle 5.1: Verwendete Hyperparameter im Vergleich zu den Referenzwerten.

Parameter	Gewählter Wert	Referenzwert (Mnih et al.)
Optimizer	Adam	RMSProp
Lernrate (α)	1×10^{-4}	2.5×10^{-4}
Diskontfaktor (γ)	0.99	0.99
Replay Buffer Größe	250.000 Transitionen	250.000 Transitionen
Batch-Größe	512	32
Target-Netzwerk Update (C)	10.000 Schritte	10.000 Schritte
Initiale Exploration	12.500 Schritte	12.500 Schritte
ϵ -Start / Ende	1.0 / 0.1	1.0 / 0.1
ϵ -Decay	250.000 Schritte	250.000 Schritte
Trainingsdauer	50.000.000 Schritte	50.000.000 Schritte

rend im Text von „200M frames“ die Rede ist, sind die x-Achsen der Ergebnisgraphen mit „Training steps (in millions)“ beschriftet und reichen ebenfalls bis 200 Millionen [5, vgl. Abb. 3]. Diese unklare Verwendung der Terminologie sprechen auch Clark et al. [48] im Anhang ihrer brandaktuellen Arbeit an und ermutigen die Forschungsgemeinschaft, die Terminologie zu standardisieren, um Missverständnisse zu vermeiden. Eine Untersuchung der nachfolgenden einflussreichen Publikationen sorgt jedoch für den ausschlaggebenden Hinweis. Die Arbeiten des Distributional-DQN [43], Meta-Gradient RL [53], IMPALA-Agenten [54], Rainbow-Agenten [45] und dem Beyond the Rainbow-Agenten [48], welche ebenfalls vergleichbare Benchmarks mit den Atari Spielen erzielen wollen, machen klar, dass 200 Millionen Frames korrekt ist und somit die publizierte Angabe von 50 Millionen Frames im Originalwerk fehlerhaft ist. Um einen relevanten Vergleich mit der etablierten Forschungspraxis zu gewährleisten, wird in dieser Arbeit daher die Benchmark-Dauer von 50 Millionen Agenten-Aktionen (entsprechend 200 Millionen Frames) übernommen. Gleichzeitig wurden gezielte Anpassungen vorgenommen, um modernen Konventionen im Deep Learning Rechnung zu tragen. Anstelle des ursprünglich verwendeten RMSProp-Optimizers wird der Adam-Optimizer [55] eingesetzt. Dieser hat sich in der breiten Praxis als Standard etabliert, da er oft eine robustere und schnellere Konvergenz als traditionelle Optimizer zeigt. Des Weiteren wird die Batch-Größe von 32 auf 512 erhöht. Moderne Hardware, wie sie für diese Arbeit zur Verfügung steht, kann größere Batches effizient verarbeiten, was zu einer genaueren Schätzung des Gradienten in jedem Lernschritt führt und das Training stabilisieren kann [51].

Die einzige verbleibende Anpassung betrifft die Lernrate, welche als einer der sensibelsten Hyperparameter gilt. Da sich sowohl der Optimizer als auch die Batch-Größe vom Original unterscheiden, wurde die Lernrate im Rahmen eines vorläufigen Experiments syste-

matisch ermittelt, wie im folgenden Abschnitt beschrieben. Das resultierende, sorgfältig abgewogene Set an Hyperparametern wird für alle experimentellen Läufe von DQN und DDQN identisch verwendet, um die zentrale Forschungsfrage dieser Arbeit unter fairen und kontrollierten Bedingungen zu untersuchen.

Ermittlung der Lernrate durch ein Vor-Experiment

Die Lernrate (α) ist einer der kritischsten Hyperparameter im Deep Reinforcement Learning. Sie bestimmt die Schrittweite, mit der die Gewichte des neuronalen Netzwerks angepasst werden, und hat einen direkten Einfluss auf die Stabilität und Konvergenzgeschwindigkeit des Trainings. Eine zu hohe Lernrate kann zu instabilen, oszillierenden Updates führen, die eine Konvergenz verhindern, während eine zu niedrige Lernrate das Training unnötig verlangsamt und das Risiko erhöht, in schlechten lokalen Optima stecken zu bleiben [51].

Da sich mehrere zentrale Komponenten des Setups (Optimizer, Batch-Größe, Netzwerkarchitektur) von der Originalarbeit [3] unterscheiden, kann die dort verwendete Lernrate nicht blind übernommen werden. Um eine fundierte und für die spezifische Problemstellung geeignete Lernrate zu finden, wird ein gezieltes Vor-Experiment durchgeführt.

Für dieses Vor-Experiment wird der einfachere DQN-Algorithmus als Baseline verwendet. Alle Hyperparameter, mit Ausnahme der zu testenden Lernrate, werden auf den in Tabelle 5.1 definierten Werten konstant gehalten. Um eine aussagekräftige Tendenz der mittelfristigen Stabilität zu erhalten, wird für jede getestete Lernrate ein einzelner, repräsentativer Trainingslauf von 5 Millionen Schritten durchgeführt. Obwohl ein einzelner Lauf keine statistisch signifikanten Aussagen über die durchschnittliche Performanz zulässt, wird dieser pragmatische Ansatz als ausreichend erachtet, um die grundsätzliche Lerndynamik und Stabilität der verschiedenen Lernraten zu vergleichen und eine fundierte Auswahl für die Haupt-Experimente zu treffen. Als primäres Evaluationskriterium diene der Verlauf der gleitenden durchschnittlichen Belohnung, geglättet über ein Fenster von 500 Episoden. Diese starke Glättung ist notwendig, um aus den stark verrauschten Rohdaten der einzelnen Episoden einen interpretierbaren Lerntrend zu extrahieren, der sowohl die Lerngeschwindigkeit als auch die Stabilität der Policy widerspiegelt.

Die Auswahl der zu testenden Lernraten orientiert sich an etablierten Werten aus der Forschungsliteratur, um den relevanten Bereich für DQN-basierte Agenten systematisch zu untersuchen. Die Spanne der potenziellen Werte wird durch zwei wegweisende Arbeiten markiert: Einerseits der Standardwert von 1×10^{-3} , wie er ursprünglich für den Adam-Optimizer vorgeschlagen wurde [55] und andererseits der deutlich niedrigere, sorgfältig abgestimmte Wert von 6.25×10^{-5} , der für den komplexen, integrierten Rainbow-Agenten mit Adam-Optimizer ermittelt wurde [45].

Da der in dieser Arbeit verwendete Agent einfacher ist als Rainbow (z.B. ohne Prioritized Replay), wird angenommen, dass der optimale Wertebereich zwischen oder in der Nähe dieser beiden Referenzpunkte liegt. Um diesen Bereich zu evaluieren, werden fünf verschiedene Lernraten in diesem Bereich getestet: 1×10^{-3} (0.001), 5×10^{-4} (0.0005), 2.5×10^{-4} (0.00025), 1×10^{-4} (0.0001) und 6.25×10^{-5} (0.0000625). Diese Auswahl ermöglicht eine Analyse von der hohen Standard-Lernrate des Adam-Optimizers [55] bis hin zu den sehr konservativen Werten, die sich in modernen, komplexen Deep-RL-Systemen als robust erwiesen haben. Es wird erwartet, dass sich ein Kompromiss zwischen sehr hohen, instabilen Lernraten und sehr niedrigen, langsam konvergierenden Raten finden lässt, der ein robustes und gleichzeitig performantes Training ermöglicht.

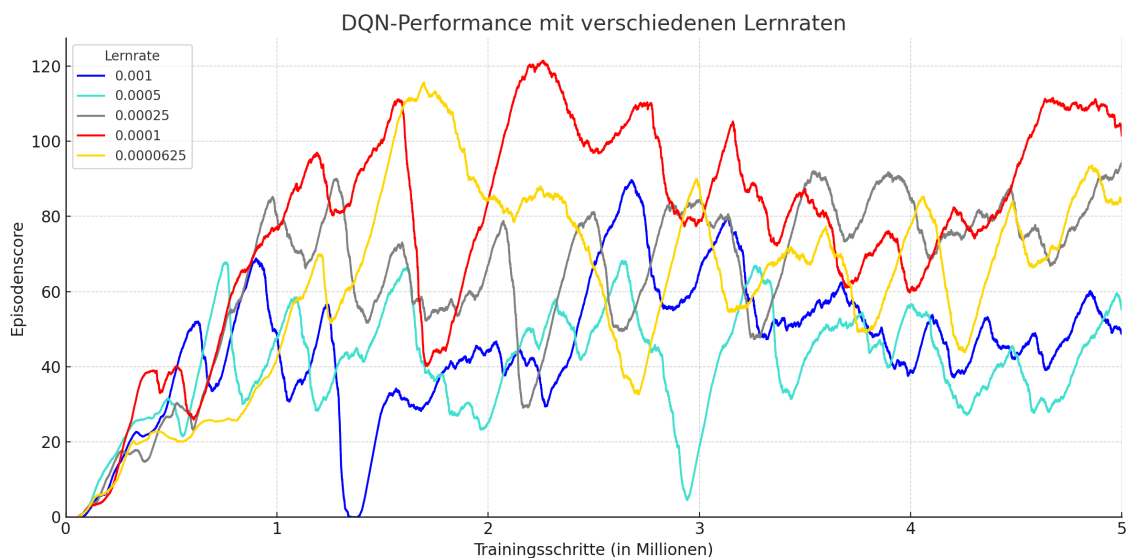


Abbildung 5.3: Vergleich der anfänglichen Lerndynamik des DQN-Agenten bei fünf verschiedenen Lernraten. Die Y-Achse zeigt den gleitenden Durchschnitt der kumulativen Belohnung pro Episode, berechnet über ein Fenster von 500 Episoden. Die X-Achse stellt den Trainingsfortschritt in Millionen von Umgebungsinteraktionen (Schritten) dar. Jede Kurve repräsentiert einen einzelnen Trainingslauf mit einem festen Seed (Seed 0) über 5 Millionen Schritte.

Die Ergebnisse des Vor-Experiments, visualisiert in Abbildung 5.3, offenbaren den signifikanten und nicht-trivialen Einfluss der Lernrate auf die Performanz des DQN-Agenten. Die Analyse der Lernkurven zeigt ein klares Spektrum von Verhaltensweisen: Die höchsten Lernraten von 1×10^{-3} (blau) und 5×10^{-4} (türkis) führen zu einer ausgeprägten Form der Trainingsinstabilität, bekannt als katastrophale Interferenz oder Vergessen [56]. Die Agenten lernen zunächst schnell eine rudimentäre Policy, die zu steigenden Scores führt. Die nachfolgenden, zu großen Gewichts-Updates überschreiben dieses fragile Wissen jedoch und führen zu einem abrupten Kollaps der Policy nach ca. 1.300.000 Schritten (blau) bzw. 2.900.000 Schritten (türkis). Der Agent verlernt dabei selbst grund-

legendste Verhaltensweisen, was sich in der Simulation durch wiederholtes, rückwärts an die Wand fahren manifestiert. Dieses Verhalten bestätigt, dass die Lernraten vermutlich zu aggressiv sind, um ein stabiles, inkrementelles Lernen zu ermöglichen. Die Lernrate von 2.5×10^{-4} (grau) zeigt zwar ein sehr stabiles Lernverhalten, erreicht jedoch nicht die höheren Leistungsbereiche, die von anderen Lernraten demonstriert werden. Die interessantesten beiden Kandidaten sind die niedrigsten Lernraten. Die Lernraten von 1×10^{-4} (rot) und 6.25×10^{-5} (gelb) zeigen die Fähigkeit, hohe Leistungsbereiche zu erreichen. Die beiden weisen jedoch eine hohe Volatilität auf, mit starken Anstiegen und deutlichen Einbrüchen. Im direkten Vergleich zeigt die Lernrate von 1×10^{-4} (rot) die größte Robustheit. Trotz sichtbarer Volatilität kehrt die Policy immer wieder zu hohen Leistungsplateaus zurück, was auf eine robuste Lernfähigkeit hindeutet. Obwohl eine gewisse Instabilität bestehen bleibt, wird für die Haupt-Experimente die Lernrate gewählt, die das höchste und persistenteste Leistungspotenzial demonstriert.

5.2.2 Statistische Absicherung

Reinforcement-Learning-Prozesse sind inhärent stochastisch, bedingt durch die standardmäßig zufällige Initialisierung der Netzwerkgewichte und die explorative Natur der ϵ -greedy-Policy. Ein einzelner Trainingslauf ist daher nicht repräsentativ. Diese hohe Varianz ist ein bekanntes Problem im Deep Reinforcement Learning und macht eine rigorose statistische Auswertung unerlässlich, um verlässliche Schlussfolgerungen zu ziehen [47]. Um dieser Anforderung gerecht zu werden, wird jeder Algorithmus (DQN und DDQN) mit fünf unabhängigen Läufen trainiert. Hierfür wird der feste Satz von Seeds [0, 1, 2, 3, 4] verwendet. Jeder Lauf verwendet einen dieser Seeds, der die Initialisierung der Netzwerkgewichte und die zufälligen Aktionen während der Exploration und die Stichprobenziehung aus dem Replay Buffer steuert und garantiert, dass die Vergleichsläufe unter identischen Bedingungen starten. Die Ergebnisse werden anschließend aggregiert, indem der Mittelwert und die Standardabweichung über die fünf Läufe berechnet werden.

Trainings-Infrastruktur und Performanz-Analyse

Alle Experimente wurden auf einem Apple MacBook Pro mit einem M4 Pro Chip durchgeführt, der über eine 14-Kern-CPU, eine 20-Kern-GPU und 48 GB Unified Memory verfügt. Um die effizienteste Hardware-Konfiguration für die Haupt-Experimente zu ermitteln, wurde ein initiales Kontrollexperiment durchgeführt. Dabei wurde die Gesamtdauer eines Trainingslaufs von 50 Millionen Schritten bei identischer Konfiguration (vgl. Tabelle 5.1) für beide Algorithmen sowohl auf der CPU als auch auf der GPU verglichen. Die Ergebnisse dieser Analyse sind in Abbildung 5.4 dargestellt.

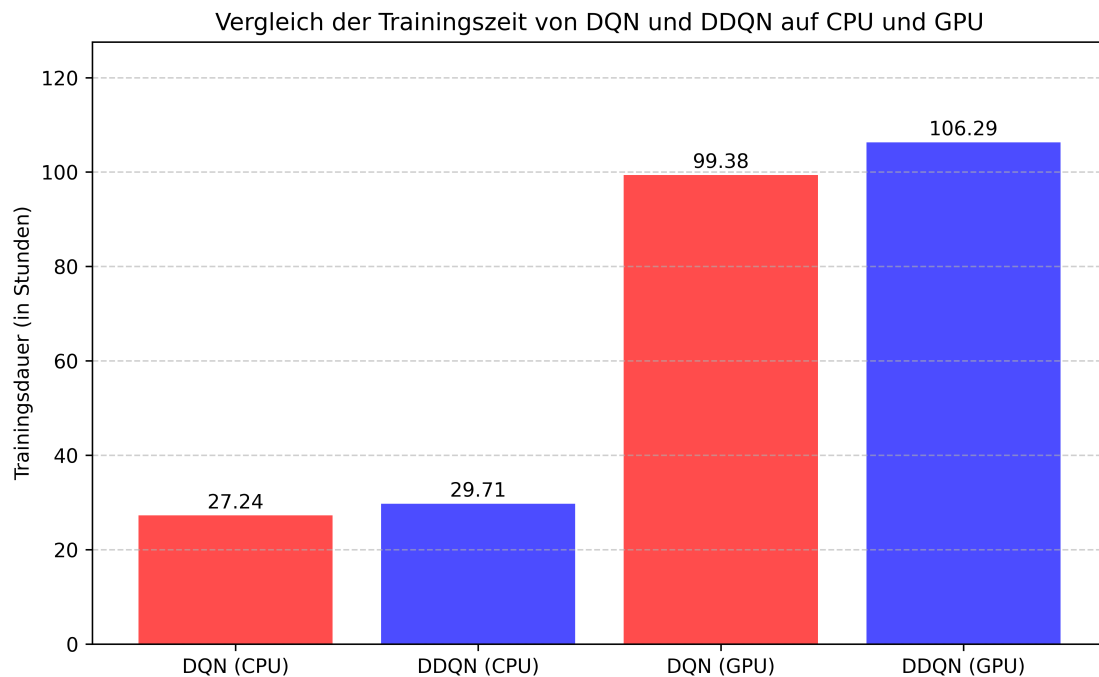


Abbildung 5.4: Vergleich der Trainingsdauer von DQN (rot) und DDQN (blau) auf CPU und GPU bei identischer Konfiguration (siehe Tabelle 5.1) über 50 Millionen Trainingsschritte mit Seed 0.

Die Analyse zeigt deutlich, dass die CPU-basierte Ausführung für diese Aufgabe erheblich performanter ist. Während ein vollständiger Trainingslauf des DQN-Agenten auf der CPU circa 27 Stunden dauerte, benötigte die Ausführung auf der GPU mit circa 99 Stunden mehr als dreieinhalbmal so lange. Auch beim DDQN-Agenten fiel das Ergebnis ähnlich aus: Etwa 30 Stunden auf der CPU standen rund 106 Stunden auf der GPU gegenüber. Weiterhin zeigt sich der erwartete Mehraufwand des DDQN-Algorithmus, der aufgrund einer zusätzlichen Netzwerkauswertung in jedem Lernschritt auf der CPU circa 9 % länger und auf der GPU circa 7 % länger benötigt als DQN.

Aufgrund dieser eindeutigen Ergebnisse wurden alle in Kapitel 7 präsentierten Haupt-Experimente auf der CPU ausgeführt. Eine detaillierte Interpretation dieses auf den ersten Blick kontraintuitiven Performanz-Unterschieds zwischen CPU und GPU erfolgt in der Diskussion in Kapitel 8.

5.3 Evaluationsmetriken

Zur Überprüfung der aufgestellten Hypothesen werden während des Trainings und der finalen Evaluation mehrere Metriken erfasst.

1. **Kumulative Belohnung pro Episode:** Dies ist die primäre Metrik zur Messung der Performanz. Sie summiert alle Belohnungen, die der Agent vom Start bis zum Ende einer Episode erhält. Ein höherer Wert deutet auf eine bessere Policy hin.
2. **Durchschnittlicher maximaler Q-Wert:** Die direkte Quantifizierung des Overestimation Bias stellt eine methodische Herausforderung dar. In den meisten komplexen Problemen kann der wahre, optimale Q-Wert nicht direkt ermittelt werden. Wäre dieser bekannt, wäre kein Reinforcement Learning notwendig, um ihn zu lernen. Anstatt einer Messung des absoluten Fehlers wird daher die *relative* Überschätzung zwischen den beiden Algorithmen verglichen. Gemäß Hypothese (H1) wird erwartet, dass DQN systematisch höhere Q-Werte als DDQN schätzt. Um dies zu überprüfen, wird in regelmäßigen Intervallen von 10.000 Trainingsschritten eine repräsentative Stichprobe von 1.000 Zuständen zufällig aus dem Replay Buffer gezogen. Für jeden dieser Zustände wird der maximale Q-Wert ($\max_a Q(s, a)$), den der jeweilige Agent mit seinen aktuellen Gewichten vorhersagt, berechnet. Der Durchschnitt dieser Werte über die gesamte Stichprobe dient als Metrik für die durchschnittliche maximale Q-Wert-Schätzung zu diesem Zeitpunkt.

Die Ergebnisse werden grafisch aufbereitet, wobei der Mittelwert der Metriken als durchgehende Linie und die Standardabweichung als schattierter Bereich dargestellt wird. Dies ermöglicht eine visuelle Einschätzung der Signifikanz der Leistungsunterschiede zwischen den beiden Algorithmen.

Kapitel 6

Implementierung

Dieses Kapitel beschreibt die technische Umsetzung des experimentellen Systems, das zur Überprüfung der in Kapitel 4.1.3 formulierten Hypothesen entwickelt wurde. Der Fokus liegt auf den zentralen Implementierungsdetails der Simulationsumgebung und der Lernalgorithmen, die für das Verständnis der experimentellen Ergebnisse von entscheidender Bedeutung sind. Die Erstellung der Simulationsumgebung selbst orientiert sich dabei strikt an der in Kapitel 5.1 erfolgten Formalisierung und erfüllt somit Anforderung F1.

6.1 Systemarchitektur im Überblick

Die Softwarearchitektur wurde modular konzipiert, um eine klare Trennung der Verantwortlichkeiten zu gewährleisten. Sie besteht aus drei Hauptkomponenten:

1. **Die Simulationsumgebung**, die die gesamte Spielphysik und die Interaktionsschnittstelle kapselt.
2. **Die Agenten**, welche die neuronale Netzwerkarchitektur, den Replay Buffer und die Lernlogik enthalten.
3. **Die Trainingssteuerung**, die den übergeordneten experimentellen Ablauf, die Datenerfassung und das Logging orchestriert.

Diese modulare Trennung ermöglicht es, die Agenten (DQN und DDQN) als austauschbare Komponenten in exakt derselben Umgebungs- und Trainingsinfrastruktur zu evaluieren, was eine grundlegende Anforderung (NF1) dieser Arbeit ist.

6.2 Implementierung der Fahrdynamik und Sensorik

Um eine relevante und nicht-triviale Steuerungsaufgabe zu schaffen, die über eine rein reaktive Navigation hinausgeht, wurde eine spezifische Fahrdynamik mit Drift-Verhalten implementiert.

6.2.1 Simulation des Drift-Verhaltens

Um das Fahrverhalten realistischer zu gestalten, wurde eine physikalische Trägheit in die Bewegungssteuerung integriert. Dies wird durch die Entkopplung der Fahrzeugausrichtung von der tatsächlichen Bewegungsrichtung realisiert. In jedem Zeitschritt werden zwei Winkel verwaltet:

- **Ausrichtungswinkel** (α_{target}): Dieser Winkel repräsentiert die Ausrichtung der Fahrzeugkarosserie und wird direkt durch die Lenk-Aktionen des Agenten beeinflusst.
- **Bewegungswinkel** (α_{travel}): Dieser Winkel beschreibt die tatsächliche Richtung des Geschwindigkeitsvektors. Er nähert sich dem Ausrichtungswinkel nur mit einer gewissen Trägheit an, die durch den `DRIFT_FACTOR` gesteuert wird.

Die Winkelgeschwindigkeit, die als Zustandsmerkmal an den Agenten übergeben wird, ist die Änderungsrate des Bewegungswinkels pro Zeitschritt:

$$\Delta\alpha_{\text{travel}} = \text{normalize}(\alpha_{\text{target}} - \alpha_{\text{travel}}) \times \text{DRIFT_FACTOR} \quad (6.1)$$

Diese Implementierung erfordert, dass der Agent vorausschauend agiert, um die Trägheit des Fahrzeugs zu kompensieren, was das Lernproblem anspruchsvoller macht als eine einfache, reaktive Steuerung und somit den Einsatz eines neuronalen Netzes weiter begründet.

6.2.2 Implementierung der Distanzsensoren

Die 16 Distanzsensoren werden durch Ray-Casting implementiert. Für jeden der vordefinierten Winkel wird ein Strahl von der Fahrzeugposition aus projiziert und der Schnittpunkt mit der nächstgelegenen Streckenbegrenzung ermittelt. Um die Berechnung zu optimieren, schließt eine räumliche Vorfilterung (Broad Phase) vorab viele Wandsegmente aus, die den Strahl aufgrund ihrer Position unmöglich schneiden können. Dies reduziert die Anzahl der teuren Linien-Schnittpunkt-Tests erheblich und ist entscheidend für eine performante Simulation.

6.3 Implementierung der Lernalgorithmen DQN und DDQN

Der Kern des Experiments liegt im direkten Vergleich der Update-Logik von DQN und DDQN. Beide Agenten teilen sich dieselbe Basisarchitektur, unterscheiden sich aber fundamental in der Implementierung ihrer `learn`-Methode. Die nachfolgenden Quellcodes veranschaulichen den Unterschied in vereinfachter Form.

6.3.1 DQN-Zielwertberechnung

Die folgende Implementierung realisiert die in Anforderung F2 geforderte Logik des Standard-DQN.

```

1 # Schritt 1: Q-Werte für Folgezustände vom Target-Netzwerk holen
2 q_next_target = self.target_network.q_eval(next_states_tensor)
3
4 # Schritt 2: Den maximalen Q-Wert für jeden Folgezustand finden
5 max_q_next = np.max(q_next_target.numpy(), axis=1)
6
7 # Schritt 3: Den finalen Zielwert berechnen
8 q_target_values = rewards + self.gamma * max_q_next * dones

```

Quellcode 6.1: Kernlogik der DQN-Zielwertberechnung

Hier wird, wie in der Theorie beschrieben, der Maximierungsoperator direkt auf die Q-Werte des Target-Netzwerks angewendet, was die Quelle des potenziellen Overestimation Bias ist.

6.3.2 DDQN-Zielwertberechnung

Die präsentierte Implementierung realisiert die in Anforderung F3 definierte Logik des DDQN-Algorithmus.

```

1 # Schritt 1: Beste Aktion fuer Folgezustände mit dem Online-Netzwerk
  auswählen
2 q_next_online = self.eval_network.q_eval(next_states_tensor)
3 best_actions = tf.argmax(q_next_online, axis=1)
4
5 # Schritt 2: Q-Werte fuer Folgezustände vom Target-Netzwerk holen
6 q_next_target = self.target_network.q_eval(next_states_tensor)
7
8 # Schritt 3: Q-Werte der besten Aktionen aus dem Target-Netzwerk extrahieren
9 batch_indices = np.arange(self.batch_size, dtype=np.int32)
10 selected_q_next = q_next_target.numpy()[batch_indices, best_actions]
11
12 # Schritt 4: Den finalen Zielwert berechnen
13 q_target_values = rewards + self.gamma * selected_q_next * dones

```

Quellcode 6.2: Kernlogik der DDQN-Zielwertberechnung

Diese Implementierung stellt sicher, dass die Aktionsauswahl (Schritt 1) und die Aktionsbewertung (Schritt 3) von unterschiedlichen Netzwerken durchgeführt werden. Dies ist der zentrale Mechanismus, der den Overestimation Bias reduzieren soll und dessen Wirksamkeit im Rahmen dieser Arbeit untersucht wird.

6.4 Steuerung des experimentellen Ablaufs

Der Haupt-Trainings-Loop, implementiert in `main.py`, stellt die strikte Vergleichbarkeit (NF1) und Reproduzierbarkeit (NF2) sicher. Vor jedem Lauf werden die Zufallsgeneratoren von Python, NumPy und TensorFlow mit einem festen Seed initialisiert. Das Skript orchestriert den sequenziellen Ablauf von Umgebungsschritt (`game.step()`), Speicherung der Transition (`agent.remember()`) und Lernschritt (`agent.learn()`). Parallel dazu werden alle relevanten Metriken, wie in NF3 gefordert, nach jeder Episode in einer CSV-Datei gespeichert und an einen TensorBoard-Writer gesendet. Diese kombinierte Fähigkeit zur Durchführung von Trainingsläufen und deren Protokollierung erfüllt die Anforderung F4. Der gesamte Prozess ist dabei performant genug, um den in NF4 gesetzten Zeitrahmen einzuhalten.

Kapitel 7

Experimente und Evaluation

Dieses Kapitel präsentiert die empirischen Ergebnisse des durchgeführten Vergleichs zwischen dem DQN- und dem DDQN-Algorithmus. Die Darstellung folgt der in Kapitel 5.3 definierten Evaluationsmethodik und zielt darauf ab, die in Kapitel 4.1.3 formulierten Hypothesen H1 und H2 zu überprüfen. Zunächst wird die allgemeine Lernperformanz beider Agenten anhand der kumulativen Belohnung analysiert. Anschließend wird die zentrale Hypothese des Overestimation Bias durch eine detaillierte Untersuchung der geschätzten Q-Werte direkt überprüft.

7.1 Analyse der Lernperformanz

Die primäre Metrik zur Bewertung der Agentenleistung ist die kumulative Belohnung pro Episode. Sie quantifiziert die Fähigkeit des Agenten, eine Policy zu erlernen, die das übergeordnete Ziel – das schnelle und erfolgreiche Absolvieren der Rennstrecke – maximiert.

7.1.1 Quantitative Analyse des Lernverhaltens

Abbildung 7.1 stellt die Lernkurven beider Agenten gegenüber. Die Ergebnisse deuten auf eine im Mittel überlegene Performanz des DDQN-Agenten hin. Insbesondere in der frühen Trainingsphase (ca. 0-10 Millionen Schritte) und in der späten Konvergenzphase (ab ca. 35 Millionen Schritten) erzielt der DDQN-Agent im Mittel höhere durchschnittliche Scores. Die Konfidenzintervalle, welche die hohe, für Reinforcement-Learning-Prozesse typische Varianz widerspiegeln [47], überlappen sich jedoch über weite Strecken des Trainings. Auffällig ist eine Phase der Annäherung im mittleren Trainingsabschnitt (ca. 16-25 Millionen Schritte), in der beide Agenten eine sehr ähnliche Leistung zeigen. Insgesamt stützen die Daten, trotz der Varianz, die Hypothese H2, welche eine verbesserte Policy durch DDQN erwartet.

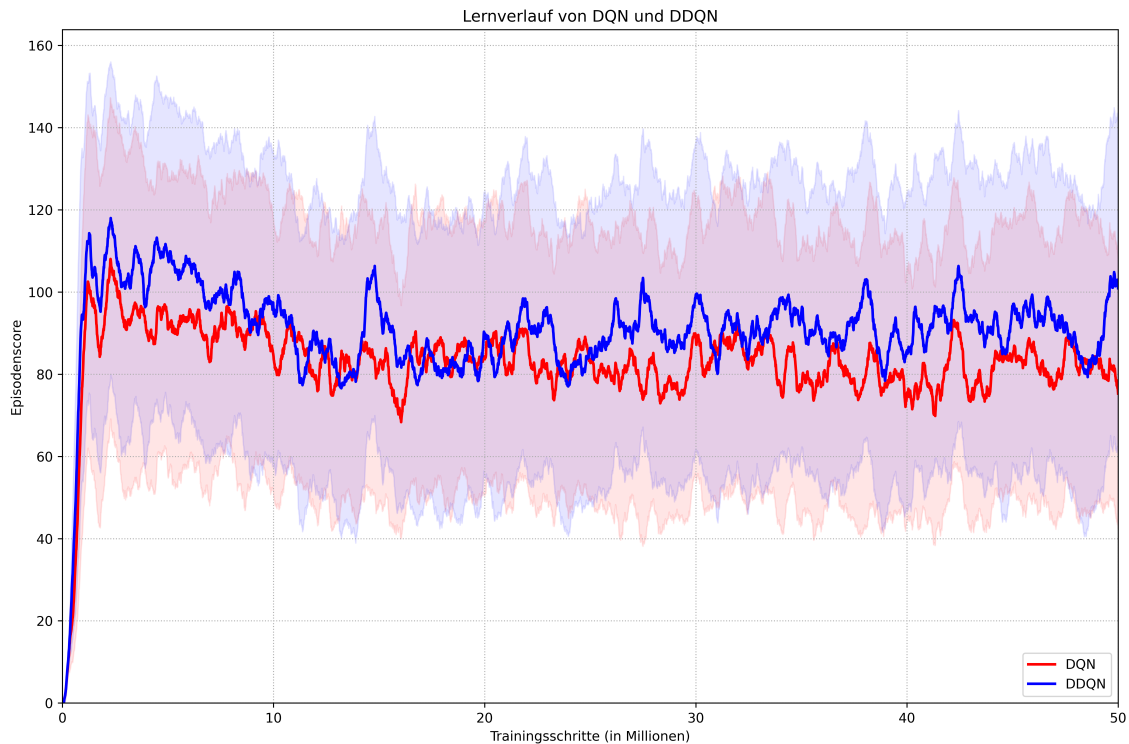


Abbildung 7.1: Vergleich der Lernperformanz von DQN (rot) und DDQN (blau). Die durchgehenden Linien zeigen den gleitenden Durchschnitt der kumulativen Belohnung pro Episode, aggregiert über fünf unabhängige Läufe. Die schattierten Bereiche stellen das 95%-Konfidenzintervall dar.

7.1.2 Qualitative Analyse des Lernverhaltens

Während Abbildung 7.1 die quantitative Lernentwicklung aggregiert darstellt, bietet ein begleitendes Video eine qualitative Visualisierung des Lernprozesses eines repräsentativen DDQN-Agenten (Seed 4). Das Video dokumentiert den sichtbaren Wandel vom anfänglich desorientierten Verhalten hin zu einer zunehmend kompetenten Fahrweise. Besondere Meilensteine, wie das erstmalige Absolvieren einer kompletten Runde, das Erreichen der maximalen Episodenlänge und schließlich eine optimierte Fahrstrategie, die in der gleichen Zeit deutlich mehr Belohnungspunkte erzielt, werden gezielt gezeigt (Passende Zeitstempel sind in der Videobeschreibung zu finden). Damit veranschaulicht es greifbar, was der Anstieg der Lernkurve in der Praxis bedeutet und wie der Agent seine Policy über die Zeit verbessert.²

²Video verfügbar unter: <https://youtu.be/y3EOL72tL3Y?si=IDFg8PGGxQSa93rZ> (besucht am 03.08.2025)

7.2 Quantifizierung des Overestimation Bias

Um die Ursache für die beobachteten Leistungsunterschiede zu untersuchen, wird Hypothese H1 direkt überprüft. Diese behauptet, dass der Standard-DQN-Algorithmus die Q-Werte systematisch überschätzt.

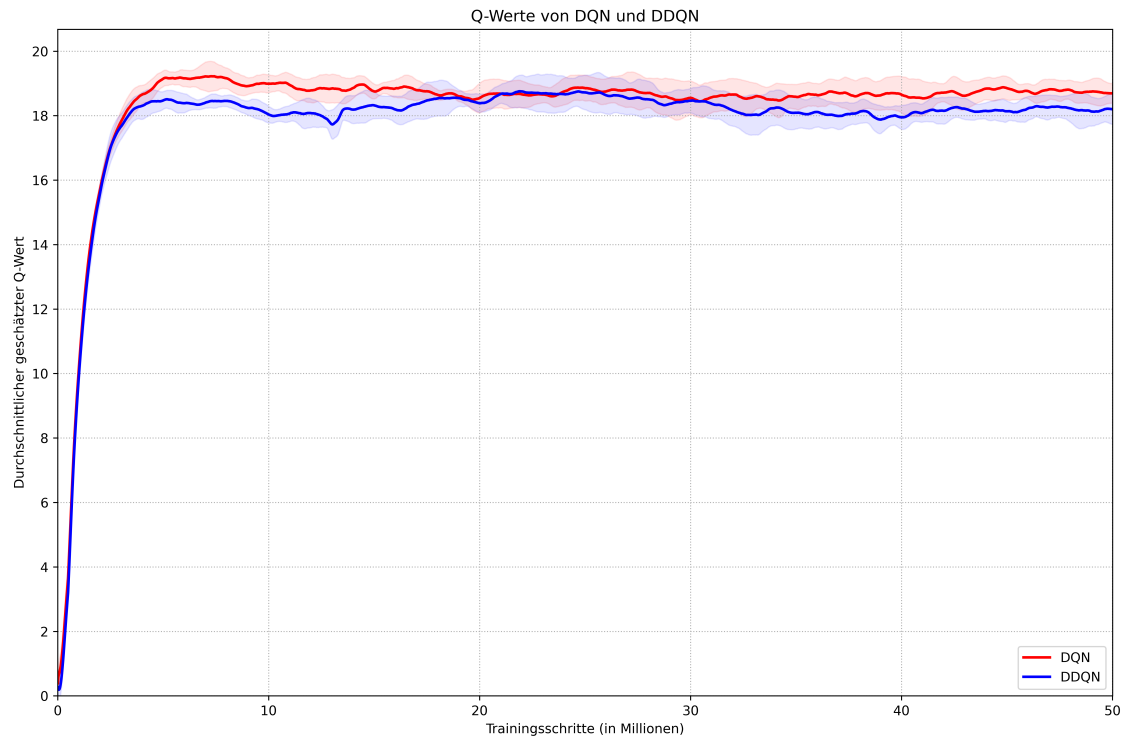


Abbildung 7.2: Vergleich des durchschnittlichen maximalen Q-Wertes von DQN (rot) und DDQN (blau). Die Werte wurden alle 10.000 Schritte über eine feste Stichprobe von 1.000 Zuständen aus dem Replay Buffer berechnet. Die durchgehenden Linien zeigen den Mittelwert über fünf Läufe, die schattierten Bereiche das 95%-Konfidenzintervall.

Abbildung 7.2 visualisiert den Verlauf der durchschnittlichen maximalen Q-Wert-Schätzungen und liefert eine nuancierte Bestätigung der Hypothese H1. Der DQN-Agent schätzt die Q-Werte über den gesamten Trainingsverlauf hinweg systematisch höher ein als der DDQN-Agent. Der Verlauf des Unterschieds lässt sich in drei Phasen gliedern: In der Phase der *frühen Divergenz*, nach einer initialen Stabilisierung, sind die Konfidenzintervalle zwischen circa 5 und 11 Millionen Schritten vollständig voneinander getrennt, was auf einen statistisch signifikanten Unterschied in dieser kritischen Lernphase hindeutet. Darauf folgt eine Phase der *mittleren Annäherung*, in der sich die Schätzungen beider Agenten deutlich annähern und über einen längeren Zeitraum fast identisch verlaufen. Gegen Ende des Trainings, in der Phase der *späten Divergenz*, vergrößert sich der Abstand zwischen den mittleren Q-Werten jedoch erneut, und die systematische Überschätzung durch DQN wird

wieder deutlicher. Eine statische Skalierung der Y-Achse wurde einer dynamischen Skalierung vorgezogen, um zu verdeutlichen, wie nahe die Q-Wert-Schätzungen der beiden Agenten beieinander liegen.

Kapitel 8

Diskussion

Dieses Kapitel interpretiert die in Kapitel 7 präsentierten Ergebnisse, ordnet sie in den wissenschaftlichen Kontext ein und analysiert kritisch die Resultate sowie die methodischen Entscheidungen. Abschließend werden die Limitationen der Arbeit erörtert und ein Ausblick auf zukünftige Forschung gegeben.

8.1 Interpretation und Einordnung der Ergebnisse

Die durchgeführten Experimente bestätigen die zentralen Hypothesen dieser Arbeit: Der DDQN-Algorithmus reduziert den Overestimation Bias im Vergleich zu Standard-DQN und erzielt dadurch eine im Mittel überlegene Lern-Performanz. Die Ergebnisse stehen somit im Einklang mit der wegweisenden Arbeit von van Hasselt et al. [5], welche die Überlegenheit von DDQN auf den Atari-2600-Benchmarks demonstrierten, sowie mit Studien aus anderen Domänen wie der mobilen Robotik, die ähnliche Vorteile belegen [6]. Die vorliegende Untersuchung erweitert diese Erkenntnisse, indem sie zeigt, dass der Vorteil von DDQN kein Artefakt hochdimensionaler, pixelbasierter Probleme ist, sondern auch in einer niedrigdimensionalen, merkmalsbasierten Umgebung fundamental greift.

Ein zentrales Ergebnis dieser Arbeit ist jedoch, dass der beobachtete Leistungsunterschied zwischen den beiden Algorithmen von eher subtiler Natur ist. Während die ursprünglichen Atari-Experimente teils dramatische Verbesserungen zeigten, ist der Vorsprung von DDQN in der hier entwickelten Rennspielumgebung zwar statistisch nachweisbar, aber in seiner absoluten Größe moderat. Diese Diskrepanz lässt sich auf die spezifischen Charakteristika der Problemstellung zurückführen, die für Standard-DQN vergleichsweise günstige Bedingungen schaffen und somit das negative Potenzial des Overestimation Bias abschwächen. Der kleine Aktionsraum limitiert die Gelegenheiten für eine fehlerhafte Maximierung [4]. Zudem ist die Umgebung vollständig deterministisch, was die massiven Überschätzungen, die van Hasselt [57] in stochastischen Umgebungen als Hauptproblem identifizierte, durch klare und wiederholbare Konsequenzen mildert. Diese günstigen Bedingungen werden durch das experimentelle Design weiter verstärkt, da im Vorexperiment eine Lernrate gewählt wurde, die bereits eine hohe Basis-Performanz für DQN sicherstellt.

Da die Umgebung, anders als in den Systemen von Barfuss & Mann [58], auch vollständig beobachtbar ist, ist das Feedback frei von Unsicherheit. Dieser klare Zusammenhang zwischen Aktion und Ergebnis wirkt als effektiver Korrekturmechanismus, der es DQN erleichtert, fehlerhafte Strategien zu korrigieren. Da DQN unter diesen optimierten Bedingungen bereits gut funktioniert, fällt der relative Vorteil von DDQN geringer aus, was die beobachtete moderate Performanzlücke erklärt. Es ist daher plausibel, dass in einer komplexeren, stochastischen Umgebung mit größerem Aktionsraum, in der die Konsequenzen von Entscheidungen unsicherer sind, der robustere Bewertungsmechanismus von DDQN zu einem deutlicheren Leistungsvorsprung führen würde.

Die Lernkurven deuten zudem auf ein Sättigungsverhalten hin. Beide Agenten erreichen ein Kompetenzniveau, auf dem die grundlegenden Fahraufgaben gemeistert werden. Weitere Optimierungen, wie die Perfektionierung der Ideallinie, führen nur noch zu marginalen Score-Verbesserungen, die von der Varianz durch die fortlaufende Exploration überdeckt werden können. Dies erklärt die temporäre Annäherung der Performanz beider Agenten im mittleren Trainingsabschnitt.

8.2 Methodologischer Beitrag und Positionierung

Über die reinen Ergebnisse hinaus leistet diese Arbeit einen methodischen Beitrag durch ihren Fokus auf kontrollierte Isolation, Zugänglichkeit und Reproduzierbarkeit. Im Gegensatz zu komplexen End-to-End-Ansätzen eliminiert das hier gewählte, merkmalsbasierte Design die Herausforderung des Repräsentationslernens. Dies ermöglicht eine klare Attribution der beobachteten Effekte auf den fundamentalen algorithmischen Unterschied zwischen DQN und DDQN, was in Systemen, die direkt von Pixeln lernen, oft schwierig ist.

Zudem demonstriert die Arbeit, dass relevante Grundlagenforschung im Bereich Reinforcement Learning nicht zwingend auf ressourcenintensive Hardware angewiesen ist. Die Feststellung, dass das Experiment auf einer Standard-CPU effizienter lief als auf einer GPU, unterstreicht ein Design, das durch die Simulation der Umgebung und nicht durch das Training des neuronalen Netzes limitiert ist. Dieser Ansatz steht im Kontrast zu hochparallelisierten Architekturen wie A3C [59], die auf maximale Skalierbarkeit abzielen, aber auch eine hohe technische Komplexität aufweisen. Das hier verfolgte, einfachere und sequenzielle Modell fördert die Zugänglichkeit und Reproduzierbarkeit von Forschung, was ein zentrales Anliegen der wissenschaftlichen Gemeinschaft ist [47]. Die explizite Auseinandersetzung mit inkonsistenten Terminologien wie „Frames“ und „Steps“, ein in der Forschung wiederholt kritisiertes Problem [48], sowie die Veröffentlichung des Quellcodes sind weitere Schritte in diese Richtung.

8.3 Limitationen und Ausblick

Die Stärken des hier verfolgten experimentellen Designs definieren gleichzeitig dessen Limitationen und weisen den Weg für zukünftige Forschung. Wie in Kapitel 8.1 analysiert, schaffen mehrere Faktoren – die deterministische Umgebung, die vollständige Beobachtbarkeit, der kleine Aktionsraum und eine für DQN bereits günstige Lernrate – Bedingungen, welche die negativen Effekte des Overestimation Bias mildern. Die dringendste Aufgabe für zukünftige Forschung ist es daher, den Algorithmenvergleich unter anspruchsvolleren Bedingungen zu wiederholen. Ein naheliegender erster Schritt wäre die *Erweiterung des Aktionsraums*, etwa durch die Einführung von feiner abgestuften Lenk- und Beschleunigungswerten. Ein zweiter, fundamentalerer Schritt ist die *Einführung von Stochastizität*, beispielsweise durch eine variable Fahrphysik, die unterschiedlichen Reifengrip simuliert, oder durch dynamische Hindernisse auf der Strecke. Es ist die zentrale Hypothese, dass in diesen Szenarien mit höherer Unsicherheit und Komplexität der robustere Schätzmechanismus von DDQN zu einem deutlicheren und praktisch relevanteren Leistungsvorteil führen wird.

Des Weiteren wurde die Lernaufgabe durch eine binäre Belohnungsstruktur (+1/-1) und klare Timeout-Bedingungen bewusst einfach gehalten. Zukünftige Arbeiten könnten den Einfluss eines ausgefeilteren *Reward Shapings* untersuchen. Eine kontinuierliche Belohnungsfunktion, die beispielsweise hohe Geschwindigkeiten auf Geraden oder die Nähe zur Ideallinie in Kurven belohnt, könnte das Lernverhalten beschleunigen und zu differenzierteren Fahrstilen führen. Ferner limitiert die minimalistische Netzwerkarchitektur potenziell die maximale Lernkapazität. Die Untersuchung, wie sich deutlich komplexere Netzwerke, etwa solche mit *Dueling-Strömen* [40], auf die Sättigungsgrenze der Performanz auswirken, ist ein vielversprechender nächster Schritt.

Schließlich bietet die hier geschaffene, validierte Basis eine ideale Testplattform, um systematisch den isolierten Nutzen weiterer zentraler DQN-Verbesserungen zu quantifizieren. Hierzu zählen beispielsweise *Prioritized Experience Replay* [42] zur Effizienzsteigerung bei seltenen, aber kritischen Ereignissen sowie die Evaluation alternativer Explorationstrategien wie *Noisy Networks* [44] als Ersatz für die hier genutzte ϵ -greedy-Methode. Diese Arbeit ist somit nicht als Endpunkt, sondern als ein reproduzierbarer Ausgangspunkt für die Erforschung komplexerer und realistischerer Szenarien im Reinforcement Learning zu verstehen.

Kapitel 9

Fazit

Diese Bachelor-Thesis hat den Overestimation Bias im Reinforcement Learning durch einen direkten, empirischen Vergleich der Algorithmen DQN und DDQN analysiert. In einer eigens entwickelten, merkmalsbasierten Rennspielumgebung wurde untersucht, ob die theoretischen Vorteile von DDQN zu einer praktisch messbaren Verbesserung der Lernleistung führen.

Die durchgeführten Experimente bestätigten die zentralen Hypothesen. Es wurde nachgewiesen, dass DDQN die systematische Überschätzung der Q-Werte im Vergleich zu DQN reduziert. Diese konservativere und genauere Werteschätzung resultierte in einer im Mittel überlegenen und stabileren Lern-Performanz, die sich in höheren durchschnittlichen Belohnungen pro Episode manifestierte. Die zentrale Forschungsfrage kann somit positiv beantwortet werden. Die Ergebnisse untermauern, dass der Vorteil von DDQN ein fundamentaler algorithmischer Mechanismus ist, dessen Wirksamkeit nicht auf hochdimensionale, pixelbasierte Probleme beschränkt ist.

Gleichzeitig offenbarte die Analyse, dass der Leistungsvorsprung in der gewählten Umgebung von subtiler Natur war. Dies wurde auf die spezifischen Eigenschaften der Problemstellung zurückgeführt: ein kleiner, diskreter Aktionsraum, eine deterministische und beobachtbare Umgebung sowie eine sorgfältig gewählte, bereits für DQN performante Lernrate. Diese Faktoren dämpfen die negativen Auswirkungen des Overestimation Bias und führen dazu, dass sich die Performanz der Algorithmen annähert, sobald ein grundlegendes Kompetenzniveau erreicht ist.

Methodologisch leistet die Arbeit einen Beitrag durch ihren Fokus auf kontrollierte Isolation und Reproduzierbarkeit. Das minimalistische Design ermöglicht eine klare Attribution der Ergebnisse und demonstriert, dass fundamentale RL-Forschung auch mit zugänglicher Standard-Hardware robust durchgeführt werden kann.

Die gewonnenen Erkenntnisse legen eine solide Grundlage für zukünftige Forschung. Ein logischer nächster Schritt ist die Untersuchung des Algorithmenvergleichs unter anspruchsvolleren Bedingungen, insbesondere in stochastischen Umgebungen und mit komplexeren Aktionsräumen. Es ist zu erwarten, dass in Szenarien mit höherer Unsicherheit die Vorteile einer robusteren Werteschätzung noch deutlicher zutage treten werden. Die fortgesetzte

Analyse und Mitigation solcher algorithmischer Feinheiten wird daher entscheidend sein, um das volle Potenzial des Reinforcement Learning für komplexe reale Anwendungen zu erschließen.

Abkürzungsverzeichnis

API	Application Programming Interface
DDPG	Deep Deterministic Policy Gradient
DDQN	Double Deep Q-Network
DQN	Deep Q-Network
DRL	Deep Reinforcement Learning
MDP	Markov Decision Process
MLP	Multi-Layer Perceptron
ReLU	Rectified Linear Unit
RL	Reinforcement Learning
SAC	Soft Actor-Critic
SVG	Scalable Vector Graphics

Glossar

Agent

Die lernende Entität, die mit der Umgebung interagiert. In dieser Arbeit das computergesteuerte Fahrzeug.

Bellman-Gleichung

Fundamentale, rekursive Beziehung, die den Wert eines Zustands durch den Wert seiner Nachfolger ausdrückt. Grundlage für wertbasierte RL-Algorithmen.

Deep Q-Network

Ein Algorithmus, der Q-Learning mit tiefen neuronalen Netzen als Funktionsapproximator kombiniert.

Double Deep Q-Network

Erweiterung von DQN zur Reduzierung des Overestimation Bias durch Entkopplung von Aktionsauswahl und -bewertung.

Drift-Verhalten

Fahrphysik mit Trägheit: Die Bewegungsrichtung folgt der Lenkausrichtung verzögert, was die Steuerung anspruchsvoller macht.

Episode

Eine vollständige Interaktionssequenz des Agenten, von einem Startzustand bis zu einem Endzustand (z.B. Kollision).

Experience Replay

Technik zum Speichern und Wiederverwenden von vergangenen Transitionen für ein stabileres und effizienteres Training.

Exploration-Exploitation-Dilemma

Der Zielkonflikt zwischen der Ausnutzung bekannten Wissens (Exploitation) und dem Erkunden neuer, potenziell besserer Strategien (Exploration).

Frame

Einzelner, gerenderter Bildschirmausschnitt. Ein Schritt kann mehrere Frames umfassen (Frame-Skipping).

Funktionsapproximation

Einsatz eines parametrisierten Modells (z.B. neuronales Netz) zur Schätzung einer Wert- oder Policy-Funktion.

Markov-Entscheidungsprozess

Mathematisches Framework für sequentielle Entscheidungsprobleme, definiert durch Zustände, Aktionen, Übergänge und Belohnungen.

Modellfrei

RL-Algorithmen, die direkt aus Interaktionen lernen, ohne ein explizites Umgebungsmodell zu erstellen.

Off-Policy

Eigenschaft von Algorithmen, aus Aktionen zu lernen, die nicht von der aktuellen Policy generiert wurden.

Overestimation Bias

Systematische Tendenz des DQN-Algorithmus, Q-Werte zu überschätzen, verursacht durch den Maximierungsoperator im Lern-Update.

Policy

Die Verhaltensweise des Agenten; eine Funktion, die Zustände auf Aktionen abbildet (π).

Q-Wert

Der erwartete Return nach Ausführung einer Aktion in einem Zustand. Quantifiziert wie gut ein Zustand-Aktions-Paar ist.

Return

Kumulierte, diskontierte Summe zukünftiger Belohnungen; die Größe, die im Reinforcement Learning maximiert wird.

Schritt

Die atomare Einheit der Interaktion zwischen Agent und Umgebung. Ein Schritt umfasst die Auswahl einer Aktion und deren Ausführung.

Seed

Initialwert für einen Pseudozufallszahlengenerator zur Gewährleistung der Reproduzierbarkeit von Experimenten.

Target-Netzwerk

Kopie des Lern-Netzwerks, die stabile Zielwerte für das Training liefert und periodisch aktualisiert wird.

Transition

Ein Datentupel, das einen Interaktionsschritt (Zustand, Aktion, Belohnung, Folgezustand) repräsentiert und im Replay Buffer gespeichert wird.

Wertbasiert

RL-Algorithmen, die eine Wertefunktion lernen und daraus ihre Policy implizit ableiten.

Abbildungsverzeichnis

2.1	Agent-Umgebung-Interaktion	6
2.2	Prozedurale Generierung der Streckengeometrie	18
5.1	Visualisierung der Distanzsensoren des Agenten	34
5.2	Visualisierung der Checkpoints auf der Rennstrecke	37
5.3	Vergleich der Lerndynamik des DQN-Agenten	42
5.4	Trainingszeit im Vergleich	44
7.1	Vergleich der kumulativen Belohnung über 5 Seeds	51
7.2	Vergleich der geschätzten Q-Werte über 5 Seeds	52

Tabellenverzeichnis

4.1	Anforderung F1 – Implementierung der Rennspielumgebung	29
4.2	Anforderung F2 – Implementierung des DQN-Agenten	29
4.3	Anforderung F3 – Implementierung des DDQN-Agenten	30
4.4	Anforderung F4 – Durchführung und Protokollierung von Trainingsläufen	30
4.5	QAS NF1 – Vergleichbarkeit der Algorithmen	31
4.6	QAS NF2 – Reproduzierbarkeit der Ergebnisse	31
4.7	QAS NF3 – Vollständigkeit der Evaluationsmetriken	32
4.8	QAS NF4 – Angemessener Zeitrahmen für das Training	32
5.1	Verwendete Hyperparameter im Vergleich zu den Referenzwerten.	40

Symbolverzeichnis

Symbol	Beschreibung
$A(s)$	Menge der verfügbaren Aktionen im Zustand s
G_t	Kumulierte, diskontierte Belohnung (Return)
$L(\theta)$	Verlustfunktion für das Netzwerk-Training
S	Menge aller möglichen Zustände (Zustandsraum)
α	Lernrate des Optimierungsalgorithmus
γ	Diskontierungsfaktor für zukünftige Belohnungen
π^*	Optimale Strategie
π	Strategie (Policy), die Aktionen auswählt
θ^-	Parameter (Gewichte) des Target-Netzwerks
θ	Parameter (Gewichte) des Online-Netzwerks
ε	Explorationsrate für die ε -greedy-Strategie
a	Aktion des Agenten
$q^*(s, a)$	Optimale Aktions-Wertefunktion
r	Unmittelbare numerische Belohnung (Reward)
s	Zustand der Umgebung
$v^*(s)$	Optimale Zustands-Wertefunktion
y_t	Zielwert (Target) für die Aktualisierung des Q-Werts

Quellcodeverzeichnis

6.1	Kernlogik der DQN-Zielwertberechnung	48
6.2	Kernlogik der DDQN-Zielwertberechnung	48

Literatur

- [1] David Silver, Aja Huang, Chris J. Maddison, Arthur Guez, Laurent Sifre, George van den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, Sander Dieleman, Dominik Grewe, John Nham, Nal Kalchbrenner, Ilya Sutskever, Timothy Lillicrap, Madeleine Leach, Koray Kavukcuoglu, Thore Graepel und Demis Hassabis. „Mastering the game of Go with deep neural networks and tree search“. In: *Nature* 529.7587 (Jan. 2016), S. 484–489. ISSN: 1476-4687. DOI: [10.1038/nature16961](https://doi.org/10.1038/nature16961).
- [2] Wojciech Samek, Thomas Wiegand und Klaus-Robert Müller. *Explainable Artificial Intelligence: Understanding, Visualizing and Interpreting Deep Learning Models*. Aug. 2017. DOI: [10.48550/arXiv.1708.08296](https://doi.org/10.48550/arXiv.1708.08296).
- [3] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dhharshan Kumaran, Daan Wierstra, Shane Legg und Demis Hassabis. „Human-level control through deep reinforcement learning“. In: *Nature* 518.7540 (Feb. 2015), S. 529–533. ISSN: 1476-4687. DOI: [10.1038/nature14236](https://doi.org/10.1038/nature14236).
- [4] Sebastian Thrun und Anton Schwartz. „Issues in Using Function Approximation for Reinforcement Learning“. In: *Proceedings of the 1993 Connectionist Models Summer School*. Lawrence Erlbaum Associates, 1993, S. 255–263. ISBN: 9781315806433.
- [5] Hado van Hasselt, Arthur Guez und David Silver. „Deep Reinforcement Learning with Double Q-Learning“. In: *Proceedings of the AAAI Conference on Artificial Intelligence* 30.1 (März 2016). ISSN: 2374-3468. DOI: [10.1609/aaai.v30i1.10295](https://doi.org/10.1609/aaai.v30i1.10295).
- [6] Min-Fan Ricky Lee und Sharfiden Hassen Yusuf. „Mobile Robot Navigation Using Deep Reinforcement Learning“. In: *Processes* 10.12 (Dez. 2022), S. 2748. ISSN: 2227-9717. DOI: [10.3390/pr10122748](https://doi.org/10.3390/pr10122748).
- [7] R.S. Sutton und A.G. Barto. „Reinforcement Learning: An Introduction“. In: *IEEE Transactions on Neural Networks* 9.5 (1998), S. 1054–1054. DOI: [10.1109/TNN.1998.712192](https://doi.org/10.1109/TNN.1998.712192).

- [8] Jörn Fischer. *Maschinelles Lernen für Dummies*. de. 1. Auflage. Wiley-VCH, Apr. 2024. ISBN: 978-3-527-72055-2.
- [9] S. B. Kotsiantis. „Supervised Machine Learning: A Review of Classification Techniques“. In: *Proceedings of the 2007 Conference on Emerging Artificial Intelligence Applications in Computer Engineering: Real Word AI Systems with Applications in EHealth, HCI, Information Retrieval and Pervasive Technologies*. 2007, S. 3–24. ISBN: 9781586037802.
- [10] Mohamed Alloghani, Dhiya Al-Jumeily, Jamila Mustafina, Abir Hussain und Ahmed J. Aljaaf. „A Systematic Review on Supervised and Unsupervised Machine Learning Algorithms for Data Science“. In: *Supervised and Unsupervised Learning for Data Science*. Hrsg. von Michael W. Berry, Azlinah Mohamed und Bee Wah Yap. Springer International Publishing, 2020, S. 3–21. ISBN: 978-3-030-22475-2. DOI: [10.1007/978-3-030-22475-2_1](https://doi.org/10.1007/978-3-030-22475-2_1).
- [11] Iqbal H. Sarker. „Machine Learning: Algorithms, Real-World Applications and Research Directions“. In: *SN Computer Science* 2.3 (März 2021), S. 160. ISSN: 2661-8907. DOI: [10.1007/s42979-021-00592-x](https://doi.org/10.1007/s42979-021-00592-x).
- [12] Iqbal H. Sarker, A. S. M. Kayes, Shahriar Badsha, Hamed Alqahtani, Paul Watters und Alex Ng. „Cybersecurity data science: an overview from machine learning perspective“. In: *Journal of Big Data* 7.1 (Juli 2020), S. 41. ISSN: 2196-1115. DOI: [10.1186/s40537-020-00318-5](https://doi.org/10.1186/s40537-020-00318-5).
- [13] L. P. Kaelbling, M. L. Littman und A. W. Moore. „Reinforcement Learning: A Survey“. In: *Journal of Artificial Intelligence Research* (1996). DOI: [10.48550/arXiv.cs/9605103](https://doi.org/10.48550/arXiv.cs/9605103).
- [14] Stuart Russell und Peter Norvig. *Artificial Intelligence, Global Edition A Modern Approach*. Pearson Deutschland, 2021, S. 1168. ISBN: 9781292401133.
- [15] Kai Arulkumaran, Marc Peter Deisenroth, Miles Brundage und Anil Anthony Bharath. *Deep Reinforcement Learning: A Brief Survey*. Aug. 2017. DOI: [10.1109/MSP.2017.2743240](https://doi.org/10.1109/MSP.2017.2743240).
- [16] Csaba Szepesvári. *Algorithms for Reinforcement Learning*. Bd. 4. Jan. 2010. DOI: [10.1007/978-3-031-01551-9](https://doi.org/10.1007/978-3-031-01551-9).
- [17] Dimitri Bertsekas. *Dynamic programming and optimal control: Volume II*. 4. Aufl. Bd. 2. Athena scientific, 2012. ISBN: 978-1-886529-44-1.
- [18] Richard Bellman. „Dynamic Programming“. In: *Science* 153.3731 (Juli 1966). Publisher: American Association for the Advancement of Science, S. 34–37. DOI: [10.1126/science.153.3731.34](https://doi.org/10.1126/science.153.3731.34).

- [19] Tommy C. Blanchard und Samuel J. Gershman. „Pure correlates of exploration and exploitation in the human brain“. In: *Cognitive, Affective & Behavioral Neuroscience* 18.1 (Feb. 2018), S. 117–126. ISSN: 1531-135X. DOI: [10.3758/s13415-017-0556-2](https://doi.org/10.3758/s13415-017-0556-2).
- [20] Sébastien Bubeck und Nicolò Cesa-Bianchi. *Regret Analysis of Stochastic and Non-stochastic Multi-armed Bandit Problems*. Nov. 2012. DOI: [10.48550/arXiv.1204.5721](https://doi.org/10.48550/arXiv.1204.5721).
- [21] Kenji Doya. „Modulators of decision making“. In: *Nature Neuroscience* 11.4 (Apr. 2008), S. 410–416. ISSN: 1546-1726. DOI: [10.1038/nn2077](https://doi.org/10.1038/nn2077).
- [22] Tobias Friedrich, Timo Kötzing, Martin S. Krejca und Amirhossein Rajabi. „Escaping Local Optima with Local Search: A Theory-Driven Discussion“. In: *Parallel Problem Solving from Nature – PPSN XVII*. Springer International Publishing, 2022, S. 442–455. ISBN: 978-3-031-14721-0. DOI: [10.1007/978-3-031-14721-0_31](https://doi.org/10.1007/978-3-031-14721-0_31).
- [23] Deepak Pathak, Pulkit Agrawal, Alexei A. Efros und Trevor Darrell. *Curiosity-driven Exploration by Self-supervised Prediction*. Mai 2017. DOI: [10.48550/arXiv.1705.05363](https://doi.org/10.48550/arXiv.1705.05363).
- [24] Peter Auer, Nicolò Cesa-Bianchi und Paul Fischer. „Finite-time Analysis of the Multiarmed Bandit Problem“. In: *Machine Learning* 47.2 (Mai 2002), S. 235–256. ISSN: 1573-0565. DOI: [10.1023/A:1013689704352](https://doi.org/10.1023/A:1013689704352).
- [25] Peter Auer. „Using confidence bounds for exploitation-exploration trade-offs“. In: *J. Mach. Learn. Res.* 3 (März 2003), S. 397–422. ISSN: 1532-4435. DOI: [10.5555/944919.944941](https://doi.org/10.5555/944919.944941).
- [26] Joshua Hare. *Dealing with Sparse Rewards in Reinforcement Learning*. 2019. DOI: [10.48550/arXiv.1910.09281](https://doi.org/10.48550/arXiv.1910.09281).
- [27] Marc G. Bellemare, Sriram Srinivasan, Georg Ostrovski, Tom Schaul, David Saxton und Remi Munos. *Unifying Count-Based Exploration and Intrinsic Motivation*. 2016. DOI: [10.48550/arXiv.1606.01868](https://doi.org/10.48550/arXiv.1606.01868).
- [28] Ronald J. Williams. „Simple statistical gradient-following algorithms for connectionist reinforcement learning“. In: *Machine Learning* 8.3 (Mai 1992), S. 229–256. ISSN: 1573-0565. DOI: [10.1007/BF00992696](https://doi.org/10.1007/BF00992696).
- [29] Zhizhou Ren, Guangxiang Zhu, Beining Han, Jianglun Chen und Chongjie Zhang. „On the Estimation Bias in Double Q-Learning“. In: (Okt. 2020). DOI: [10.48550/arXiv.2109.14419](https://doi.org/10.48550/arXiv.2109.14419).

- [30] Christopher J. C. H. Watkins und Peter Dayan. „Q-learning“. In: *Machine Learning* 8.3 (Mai 1992), S. 279–292. ISSN: 1573-0565. DOI: [10.1007/BF00992698](https://doi.org/10.1007/BF00992698).
- [31] Jan Diekhoff und Jörn Fischer. *Recursive Backwards Q-Learning in Deterministic Environments*. Apr. 2024. DOI: [10.48550/arXiv.2404.15822](https://doi.org/10.48550/arXiv.2404.15822).
- [32] Yann LeCun, Yoshua Bengio und Geoffrey Hinton. „Deep learning“. In: *Nature* 521.7553 (Mai 2015), S. 436–444. ISSN: 1476-4687. DOI: [10.1038/nature14539](https://doi.org/10.1038/nature14539).
- [33] Long-Ji Lin. „Self-improving reactive agents based on reinforcement learning, planning and teaching“. In: *Machine Learning* 8.3 (Mai 1992), S. 293–321. ISSN: 1573-0565. DOI: [10.1007/BF00992699](https://doi.org/10.1007/BF00992699).
- [34] Martín Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, Manjunath Kudlur, Josh Levenberg, Rajat Monga, Sherry Moore, Derek G. Murray, Benoit Steiner, Paul Tucker, Vijay Vasudevan, Pete Warden, Martin Wicke, Yuan Yu und Xiaoqiang Zheng. *TensorFlow: A system for large-scale machine learning*. 2016. DOI: [10.48550/arXiv.1605.08695](https://doi.org/10.48550/arXiv.1605.08695).
- [35] Maximilian Jaritz, Raoul de Charette, Marin Toromanoff, Etienne Perot und Fawzi Nashashibi. *End-to-End Race Driving with Deep Reinforcement Learning*. Aug. 2018. DOI: [10.48550/arXiv.1807.02371](https://doi.org/10.48550/arXiv.1807.02371).
- [36] Daniele Loiacaono, Luigi Cardamone und Pier Luca Lanzi. *Simulated Car Racing Championship: Competition Software Manual*. Apr. 2013. DOI: [10.48550/arXiv.1304.1672](https://doi.org/10.48550/arXiv.1304.1672).
- [37] Xinlei Pan, Yurong You, Ziyang Wang und Cewu Lu. *Virtual to Real Reinforcement Learning for Autonomous Driving*. Apr. 2017. DOI: [10.48550/arXiv.1704.03952](https://doi.org/10.48550/arXiv.1704.03952).
- [38] Sagar Pathak, Bidhya Shrestha und Kritish Pahi. *Enhanced Deep Q-Learning for 2D Self-Driving Cars: Implementation and Evaluation on a Custom Track Environment*. Feb. 2024. DOI: [10.48550/arXiv.2402.08780](https://doi.org/10.48550/arXiv.2402.08780).
- [39] Siyu Lu, Ye Tao, Junwei Zeng und Qihuan Zuo. „A Study on the Impact of Obstacle Size on Training Models Based on DQN and DDQN“. In: *ITM Web of Conferences* 73 (2025). ISSN: 2271-2097. DOI: [10.1051/itmconf/20257301004](https://doi.org/10.1051/itmconf/20257301004).
- [40] Ziyu Wang, Tom Schaul, Matteo Hessel, Hado van Hasselt, Marc Lanctot und Nando de Freitas. *Dueling Network Architectures for Deep Reinforcement Learning*. Apr. 2016. DOI: [10.48550/arXiv.1511.06581](https://doi.org/10.48550/arXiv.1511.06581).

- [41] Oron Anschel, Nir Baram und Nahum Shimkin. *Averaged-DQN: Variance Reduction and Stabilization for Deep Reinforcement Learning*. März 2017. DOI: [10.48550/arXiv.1611.01929](https://doi.org/10.48550/arXiv.1611.01929).
- [42] Tom Schaul, John Quan, Ioannis Antonoglou und David Silver. *Prioritized Experience Replay*. 2016. DOI: [10.48550/arXiv.1511.05952](https://doi.org/10.48550/arXiv.1511.05952). arXiv: 1511.05952 [cs.LG].
- [43] Marc G. Bellemare, Will Dabney und Rémi Munos. *A Distributional Perspective on Reinforcement Learning*. Juli 2017. DOI: [10.48550/arXiv.1707.06887](https://doi.org/10.48550/arXiv.1707.06887).
- [44] Meire Fortunato, Mohammad Gheshlaghi Azar, Bilal Piot, Jacob Menick, Ian Osband, Alex Graves, Vlad Mnih, Remi Munos, Demis Hassabis, Olivier Pietquin, Charles Blundell und Shane Legg. *Noisy Networks for Exploration*. 2019. DOI: [10.48550/arXiv.1706.10295](https://doi.org/10.48550/arXiv.1706.10295).
- [45] Matteo Hessel, Joseph Modayil, Hado van Hasselt, Tom Schaul, Georg Ostrovski, Will Dabney, Dan Horgan, Bilal Piot, Mohammad Azar und David Silver. *Rainbow: Combining Improvements in Deep Reinforcement Learning*. Okt. 2017. DOI: [10.48550/arXiv.1710.02298](https://doi.org/10.48550/arXiv.1710.02298).
- [46] Adrià Puigdomènech Badia, Bilal Piot, Steven Kapturowski, Pablo Sprechmann, Alex Vitvitskyi, Daniel Guo und Charles Blundell. *Agent57: Outperforming the Atari Human Benchmark*. 2020. DOI: [10.48550/arXiv.2003.13350](https://doi.org/10.48550/arXiv.2003.13350).
- [47] Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup und David Meger. *Deep Reinforcement Learning that Matters*. Sep. 2017. DOI: [10.48550/arXiv.1709.06560](https://doi.org/10.48550/arXiv.1709.06560).
- [48] Tyler Clark, Mark Towers, Christine Evers und Jonathon Hare. *Beyond The Rainbow: High Performance Deep Reinforcement Learning on a Desktop PC*. Mai 2025. DOI: [10.48550/arXiv.2411.03820](https://doi.org/10.48550/arXiv.2411.03820).
- [49] B. Ravi Kiran, Ibrahim Sobh, Victor Talpaert, Patrick Mannion, Ahmad A. Al Sallab, Senthil Yogamani und Patrick Pérez. *Deep Reinforcement Learning for Autonomous Driving: A Survey*. Feb. 2020. DOI: [10.48550/arXiv.2002.00444](https://doi.org/10.48550/arXiv.2002.00444).
- [50] Shai Shalev-Shwartz, Shaked Shammah und Amnon Shashua. *Safe, Multi-Agent, Reinforcement Learning for Autonomous Driving*. Okt. 2016. DOI: [10.48550/arXiv.1610.03295](https://doi.org/10.48550/arXiv.1610.03295).
- [51] Ian Goodfellow, Yoshua Bengio und Aaron Courville. *Deep Learning*. MIT Press, 2016. ISBN: 9780262035613.

- [52] Timothy P. Lillicrap, Jonathan J. Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver und Daan Wierstra. *Continuous control with deep reinforcement learning*. Sep. 2015. DOI: [10.48550/arXiv.1509.02971](https://doi.org/10.48550/arXiv.1509.02971).
- [53] Zhongwen Xu, Hado van Hasselt und David Silver. *Meta-Gradient Reinforcement Learning*. Mai 2018. DOI: [10.48550/arXiv.1805.09801](https://doi.org/10.48550/arXiv.1805.09801).
- [54] Lasse Espeholt, Hubert Soyer, Remi Munos, Karen Simonyan, Volodymyr Mnih, Tom Ward, Yotam Doron, Vlad Firoiu, Tim Harley, Iain Dunning, Shane Legg und Koray Kavukcuoglu. *IMPALA: Scalable Distributed Deep-RL with Importance Weighted Actor-Learner Architectures*. Juni 2018. DOI: [10.48550/arXiv.1802.01561](https://doi.org/10.48550/arXiv.1802.01561).
- [55] Diederik P. Kingma und Jimmy Ba. *Adam: A Method for Stochastic Optimization*. Dez. 2014. DOI: [10.48550/arXiv.1412.6980](https://doi.org/10.48550/arXiv.1412.6980).
- [56] Michael McCloskey und Neal J. Cohen. „Catastrophic Interference in Connectionist Networks: The Sequential Learning Problem“. In: *Psychology of Learning and Motivation*. Hrsg. von Gordon H. Bower. Bd. 24. Academic Press, Jan. 1989, S. 109–165. DOI: [10.1016/S0079-7421\(08\)60536-8](https://doi.org/10.1016/S0079-7421(08)60536-8).
- [57] Hado Hasselt. „Double Q-learning“. In: *Advances in Neural Information Processing Systems*. Hrsg. von J. Lafferty, C. Williams, J. Shawe-Taylor, R. Zemel und A. Culotta. Bd. 23. Curran Associates, Inc., 2010.
- [58] Wolfram Barfuss und Richard P. Mann. „Modeling the effects of environmental and perceptual uncertainty using deterministic reinforcement learning dynamics with partial observability“. In: *Physical Review E* 105.3 (März 2022), S. 034409. DOI: [10.1103/PhysRevE.105.034409](https://doi.org/10.1103/PhysRevE.105.034409).
- [59] Volodymyr Mnih, Adrià Puigdomènech Badia, Mehdi Mirza, Alex Graves, Timothy P. Lillicrap, Tim Harley, David Silver und Koray Kavukcuoglu. *Asynchronous Methods for Deep Reinforcement Learning*. Juni 2016. DOI: [10.48550/arXiv.1602.01783](https://doi.org/10.48550/arXiv.1602.01783).